

Autora: Mireia Consarnau Pallarés.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Aprendizaje supervisado. Modelos para análisis predictivo.....	1
Introducción.....	1
Lección 1. ¿Qué es el aprendizaje supervisado? Fases de trabajo.....	1
Definición de aprendizaje supervisado, diferencias con programación tradicional.....	2
Tipos de problemas, clasificación y regresión.....	2
Cómo se desarrollan las fases del aprendizaje supervisado.....	3
División de datos, entrenamiento, validación, prueba, validación cruzada.....	4
Concepto de modelo, algoritmo, parámetros, hiperparámetros.....	5
Riesgos habituales, sobreajuste, subajuste, bias-variance.....	6
Lección 2. Modelos supervisados para clasificación: algoritmos, métricas y usos.....	8
Papel de la clasificación, ejemplos de aplicación práctica.....	8
Métricas para evaluar clasificación.....	8
Clasificación binaria y multiclase.....	8
Matriz de confusión para clasificación binaria.....	9
Matriz de confusión para clasificación multiclase.....	10
Exactitud (Accuracy).....	11
Precisión (Precision).....	11
Sensibilidad (Recall).....	11
Especificidad.....	11
F1-Score.....	11
Índice Kappa.....	12
Valor predictivo positivo (PPV).....	12
Valor predictivo negativo (NPV).....	12
Curva ROC y Área bajo la curva (AUC).....	12
Elección de métricas según el contexto del problema.....	13
Detección del sobreajuste en clasificación y análisis visual.....	14
Validación cruzada.....	14
Comparación entre entrenamiento y test.....	15
Curva de aprendizaje.....	16
Validación con datos externos o nuevas observaciones.....	16
Algoritmos de clasificación.....	17
Introducción a los algoritmos de clasificación.....	17
Naive Bayes fundamentos probabilísticos y supuestos.....	17
Idea principal y ejemplos de uso.....	17
Cómo funciona.....	17
Puntos clave del algoritmo.....	18
Cuándo funciona bien y cuándo no.....	18
Particularidades del algoritmo.....	19
K-Nearest Neighbors distancia y maldición de la dimensionalidad.....	19
Idea principal y ejemplos de uso.....	19
Cómo funciona.....	20

Puntos clave del algoritmo.....	21
Cuándo funciona bien y cuándo no.....	21
Particularidades del algoritmo.....	21
Regresión logística decisión lineal y regularización.....	22
Idea principal y ejemplos de uso.....	22
Cómo funciona.....	22
Puntos clave del algoritmo.....	24
Cuándo funciona bien y cuándo no.....	24
Particularidades del algoritmo.....	24
Support Vector Machines margen máximo y kernels.....	25
Idea principal y ejemplos de uso.....	25
Cómo funciona.....	25
Puntos clave del algoritmo.....	27
Cuándo funciona bien y cuándo no.....	27
Particularidades del algoritmo.....	27
Árboles de decisión criterios de división y sobreajuste.....	29
Idea principal y ejemplos de uso.....	29
Cómo funciona.....	30
Puntos clave del algoritmo.....	31
Cuándo funciona bien y cuándo no.....	31
Particularidades del algoritmo.....	31
Random Forest agregación de modelos e importancia de variables.....	32
Idea principal y ejemplos de uso.....	32
Cómo funciona.....	32
Puntos clave del algoritmo.....	33
Cuándo funciona bien y cuándo no.....	33
Particularidades del algoritmo.....	33
AdaBoost ponderación de errores y mejora iterativa.....	34
Idea principal y ejemplos de uso.....	34
Cómo funciona.....	35
Puntos clave del algoritmo.....	35
Cuándo funciona bien y cuándo no.....	36
Particularidades del algoritmo.....	36
Comparación general de algoritmos.....	37
Lección 3. Modelos supervisados para regresión: algoritmos, métricas y usos.....	40
Papel de la regresión, ejemplos de aplicación práctica.....	40
Métricas para evaluar regresión.....	40
MAE, MSE, RMSE, comparación y sensibilidad a valores extremos.....	40
MAE (Error Absoluto Medio).....	40
MSE (Error Cuadrático Medio).....	41
RMSE (Raíz del Error Cuadrático Medio).....	41
R ² y coeficiente de determinación, interpretación y límites.....	42
Funciones de pérdida, error absoluto, error cuadrático, entropía cruzada adaptada	43
Error absoluto (EAM, Error Absoluto Medio).....	43

Error cuadrático (EQM, Error Cuadrático Medio).....	43
Entropía cruzada adaptada.....	44
Comprender la diferencia entre precisión y exactitud en regresión.....	44
Detección del sobreajuste en clasificación y análisis visual.....	46
Evaluación directa de las predicciones.....	47
Gráfico de predicciones frente a valores reales.....	48
Gráfico de residuos.....	48
Curvas de aprendizaje en regresión.....	49
Validación con nuevas observaciones.....	50
Algoritmos de regresión.....	51
Introducción a los algoritmos de regresión.....	51
Regresión lineal simple y múltiple, mínimos cuadrados y regularización.....	52
Idea principal y ejemplos de uso.....	52
Cómo funciona.....	52
Puntos clave del algoritmo.....	53
Cuándo funciona bien y cuándo no.....	53
Particularidades del algoritmo.....	54
Support Vector Regression, margen ϵ y kernels.....	55
Árboles de decisión para regresión, partición de varianza.....	56
Random Forest para regresión, ensamble y reducción de varianza.....	57
Adaboost para regresión, errores ponderados continuos.....	58
Comparación general de algoritmos.....	59

Aprendizaje supervisado. Modelos para análisis predictivo

Introducción

El aprendizaje supervisado parte de la idea de usar datos con ejemplos conocidos para construir modelos que permitan hacer predicciones con datos nuevos. Esta unidad explica cómo se prepara y se organiza la información, cómo se dividen los datos en entrenamiento, validación y prueba, y cómo se revisa si un modelo generaliza bien o no.

Se verán las dos principales áreas que abarca el aprendizaje supervisado, la clasificación y la regresión. En clasificación se estudia cómo asignar a cada dato una categoría o etiqueta, mientras que en regresión se trabaja con valores numéricos continuos. También se presentarán las métricas más importantes que se utilizan para medir el rendimiento y la fiabilidad de estos modelos.

Por último se explicarán los algoritmos más clásicos y utilizados para abordar estos problemas. Entre ellos destacan Naive Bayes, k-Nearest Neighbors, regresión logística, Support Vector Machines, árboles de decisión, Random Forest y Adaboost, tanto en clasificación como en regresión.

Al finalizar, el alumno tendrá una visión clara y ordenada de cómo se desarrolla un proyecto supervisado y cómo se comparan los modelos con las métricas más comunes.

Lección 1. ¿Qué es el aprendizaje supervisado?

Fases de trabajo

Definición de aprendizaje supervisado, diferencias con programación tradicional

El aprendizaje supervisado es un método que consiste en enseñar a un modelo a realizar predicciones a partir de ejemplos conocidos. Cada ejemplo incluye las variables que describen una situación y el resultado que se espera. Durante el proceso de aprendizaje, el modelo busca patrones y relaciones entre las variables de entrada y la salida que se quiere predecir. Una vez entrenado, puede aplicar ese conocimiento a datos nuevos y predecir resultados con un grado de fiabilidad.

Esta forma de trabajar contrasta con la programación tradicional, en la que se escriben instrucciones paso a paso para que el sistema siga reglas fijas. En el aprendizaje supervisado no se programan reglas concretas, sino que el modelo las aprende de los datos. Esto hace que pueda adaptarse a problemas complejos o situaciones donde sería muy difícil o imposible escribir las reglas manualmente.

Además, el aprendizaje supervisado tiene la ventaja de que puede seguir aprendiendo y mejorando a medida que se dispone de más datos. Mientras que en la programación tradicional los cambios suelen requerir reescribir parte del código, en un modelo supervisado basta con actualizar los datos y reentrenarlo para que siga funcionando bien.

Tipos de problemas, clasificación y regresión

En el aprendizaje supervisado se pueden encontrar dos grandes tipos de problemas según el tipo de salida que se quiere predecir. Por un lado está la clasificación, que se usa cuando la variable objetivo tiene un número limitado de valores o clases. Un ejemplo sería clasificar correos electrónicos como "spam" o "no spam". Cada observación se asocia a una de estas clases, y el objetivo del modelo es aprender a predecir correctamente a cuál pertenece cada nueva observación.

Por otro lado está la regresión, que se emplea cuando la variable objetivo es numérica y continua. En este caso el modelo no decide entre clases, sino que calcula un valor concreto. Un ejemplo sería predecir el precio de una casa en función de sus características como el tamaño, la ubicación o la antigüedad. El modelo busca identificar la relación entre las variables y ajustar sus predicciones lo más cerca posible de los valores reales.

Además, existen problemas que pueden plantearse como clasificación o regresión, dependiendo de cómo se interprete la variable objetivo. Por ejemplo, predecir la edad exacta de una persona sería un problema de regresión, pero si la agrupamos en intervalos como "menor de 20", "20 a 40" o "más de 40", se convierte en un problema de clasificación.

Ambos tipos de problemas comparten la idea de aprender de datos conocidos, pero difieren en cómo interpretan el resultado final. La clasificación se centra en etiquetas o categorías, mientras la regresión se orienta a valores numéricos que pueden variar en un rango continuo.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Cómo se desarrollan las fases del aprendizaje supervisado

El aprendizaje supervisado sigue un ciclo completo que comienza mucho antes de que un modelo haga predicciones y termina con una revisión a fondo de si ese modelo funciona bien y es capaz de generalizar a datos nuevos. La fase de recopilación de datos es la primera y consiste en reunir información suficiente, relevante y actualizada. Los datos deben representar bien el problema, ya que si tienen sesgos o errores, el modelo también aprenderá esos fallos.

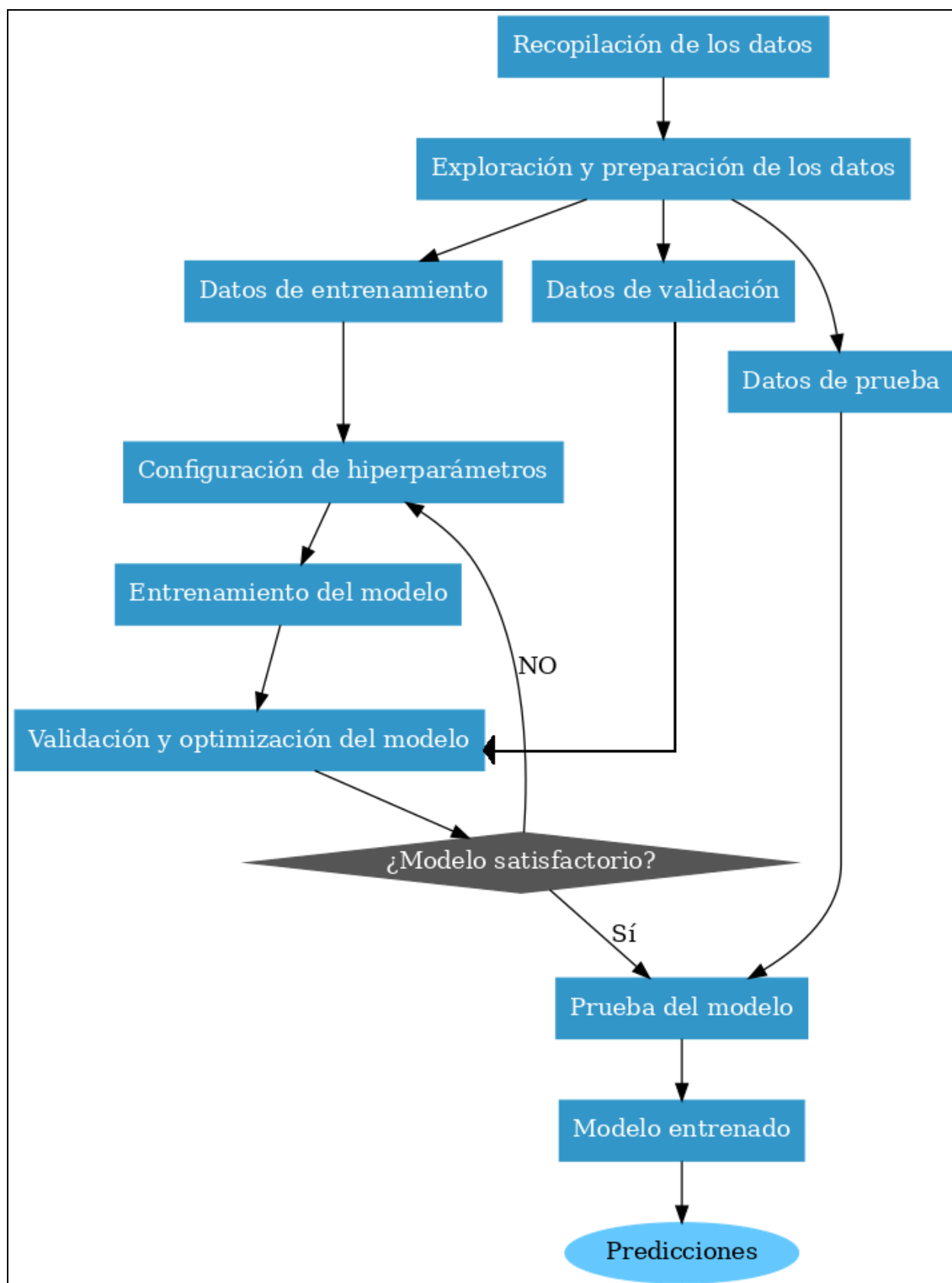
A continuación, está la fase de exploración y preparación de los datos, en la que se revisa su calidad y se hacen transformaciones básicas para dejarlos listos. Esta fase, que ya se trabajó en la unidad anterior, incluye la limpieza de valores nulos o duplicados, la corrección de datos mal introducidos, la normalización o estandarización de las escalas y la codificación de variables categóricas. Después de la limpieza, se exploran las distribuciones y las relaciones entre variables, lo que ayuda a entender los datos y detectar posibles problemas, como valores extremos o correlaciones inesperadas.

La fase de división de los datos permite separar la información en conjuntos de entrenamiento, validación y prueba. Esto es esencial para que el modelo no aprenda de memoria los datos de entrenamiento y se pueda comprobar si generaliza bien a datos que nunca ha visto. Tras la división, se pasa a la fase de entrenamiento del modelo y realización de las predicciones, donde el modelo ajusta sus parámetros internos basándose en los datos de entrenamiento.

Luego viene la fase de validación, en la que se comprueba con datos distintos a los de entrenamiento si el modelo está funcionando correctamente o si está sobreajustando. Después de esta validación, se entra en la fase de optimización, ajustando hiperparámetros, probando otros algoritmos o combinando técnicas para mejorar el rendimiento. Estas fases de validación y optimización no siempre son lineales y pueden repetirse varias veces hasta encontrar la combinación más adecuada.

Una vez elegido el modelo que mejor funciona, se realizan pruebas finales para asegurarse de que no hay sobreajuste y de que el modelo sigue funcionando bien incluso con valores extremos o situaciones poco comunes. Esto confirma que el modelo es estable y puede adaptarse a datos reales más allá del conjunto de entrenamiento.

Finalmente, la fase de conclusiones permite recoger los resultados, interpretar su significado y decidir cómo aplicar el modelo o cómo comunicar sus predicciones a quienes lo van a usar. Todo este ciclo muestra que el aprendizaje supervisado no es solo cuestión de entrenar modelos, sino de preparar y revisar los datos en cada paso para asegurar que las predicciones sean lo más precisas y fiables posible.



Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



División de datos, entrenamiento, validación, prueba, validación cruzada

Una parte esencial en el aprendizaje supervisado es cómo se dividen los datos para que el modelo pueda aprender de forma equilibrada y se pueda comprobar su rendimiento real. Lo habitual es separar el conjunto original en tres partes: datos de entrenamiento, de validación y de prueba. Una división muy común es usar un 70 % de los datos para entrenar y el 30 % restante para validación y prueba. Esta proporción puede variar según el problema, pero siempre se busca que el conjunto de entrenamiento tenga suficiente información para que el modelo pueda ajustar sus parámetros y, más adelante, sus hiperparámetros.

El conjunto de entrenamiento permite que el modelo aprenda las relaciones entre las variables de entrada y la salida. El conjunto de validación sirve para ajustar los hiperparámetros y comprobar que el modelo funciona bien con datos que no ha visto antes, ayudando a detectar problemas como el sobreajuste. El conjunto de prueba es el paso final, donde se evalúa el rendimiento real del modelo antes de aplicarlo a datos futuros.

Además de esta división tradicional, muchas veces se usa la validación cruzada para aprovechar al máximo los datos disponibles y tener una evaluación más completa del modelo. Consiste en dividir el conjunto de datos varias veces en partes diferentes, llamadas “pliegues” o “folds”. Por ejemplo, en una validación cruzada de 5 pliegues, el modelo se entrena cinco veces, cada vez dejando fuera un quinto de los datos como conjunto de validación y usando los cuatro quintos restantes para entrenamiento. Así, se obtiene una media del rendimiento en distintas divisiones, lo que ayuda a ver cómo se comporta el modelo en general y no solo con una partición concreta.

La validación cruzada es especialmente útil cuando se dispone de pocos datos o cuando se quiere tener una medida más robusta de la capacidad real del modelo para generalizar. Todo este proceso asegura que el modelo no solo funcione bien con los datos conocidos, sino que también pueda adaptarse a datos nuevos y seguir siendo útil en la práctica.

Concepto de modelo, algoritmo, parámetros, hiperparámetros

En el aprendizaje supervisado es importante distinguir bien entre varios conceptos que suelen confundirse. El modelo es la representación final que se entrena con los datos y que, una vez listo, se utiliza para hacer predicciones. Por ejemplo, un árbol de decisión entrenado con datos de pacientes para decidir si tienen o no una enfermedad concreta.

Por su parte, el algoritmo es el conjunto de pasos o instrucciones que se siguen para construir ese modelo a partir de los datos. En el caso de un árbol de decisión, el algoritmo define cómo dividir los datos y qué criterios usar para crear las ramas.

El modelo ajusta sus parámetros durante el entrenamiento. Estos son los valores internos que el modelo adapta automáticamente a medida que aprende de los datos. Por ejemplo, en la regresión lineal serían los coeficientes que multiplican las variables de entrada para dar la predicción. En un árbol de decisión serían las condiciones concretas de división en cada nodo.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Los hiperparámetros son diferentes porque no se aprenden directamente de los datos. Se establecen antes de entrenar y controlan cómo funciona el algoritmo en general. En el caso de un árbol, un hiperparámetro sería la profundidad máxima permitida o el número mínimo de muestras para hacer una división. A diferencia de los parámetros internos, estos hiperparámetros no cambian durante el entrenamiento: los define la persona que entrena el modelo o se buscan de forma automática con técnicas de búsqueda.

Para encontrar los valores adecuados de los hiperparámetros se suele usar una búsqueda sistemática, como la búsqueda en cuadrícula (grid search), que prueba distintas combinaciones posibles, o la búsqueda aleatoria (random search), que elige combinaciones de manera más flexible. Esta búsqueda ayuda a mejorar el rendimiento final sin que el modelo se adapte demasiado a los datos de entrenamiento.

Entender esta diferencia entre parámetros e hiperparámetros ayuda a ver qué valores puede aprender el modelo y cuáles son decisiones previas que pueden marcar la diferencia entre un modelo que generaliza bien y otro que se queda corto.

Riesgos habituales, sobreajuste, subajuste, bias-variance

Al trabajar con modelos supervisados es habitual encontrarse con problemas como el sobreajuste y el subajuste. El sobreajuste ocurre cuando el modelo se adapta demasiado a los datos de entrenamiento y aprende incluso detalles irrelevantes o ruido. Por ejemplo, si se entrena un árbol de decisión muy profundo, puede memorizar cada dato exacto y funcionar muy bien en el entrenamiento pero fallar con datos nuevos. Esto hace que el modelo parezca muy preciso pero en realidad no generaliza bien.

Por el contrario, el subajuste sucede cuando el modelo no es capaz de captar los patrones principales de los datos, como si se entrenara un árbol de decisión muy poco profundo o una regresión lineal para un problema muy complejo. En este caso, el modelo comete muchos errores incluso con los datos de entrenamiento y tampoco predice bien en datos nuevos.

Estos dos problemas están muy relacionados con el equilibrio entre bias y variance. El bias mide el error que se produce cuando se simplifica demasiado el problema, lo que hace que el modelo no logre captar los patrones importantes y tienda a subajustarse. La variance mide cuánto varían las predicciones del modelo con diferentes datos, y cuando es muy alta hace que el modelo se sobreajuste y se vuelva muy sensible a cambios pequeños.

Para ver si un modelo sufre sobreajuste o subajuste se pueden comparar los resultados de entrenamiento y de validación. Por ejemplo, si el error en entrenamiento es muy bajo pero en validación es muy alto, hay sobreajuste. Si los errores son altos en ambos, el modelo está subajustado. Estos resultados se suelen ver en gráficos de aprendizaje o tablas de métricas.

Existen técnicas básicas para controlar y detectar el sobreajuste, como la validación cruzada, el uso de regularización para que el modelo no se ajuste en exceso a los datos de entrenamiento y la comparación de métricas en conjuntos distintos. Además, se pueden utilizar gráficos como las curvas de aprendizaje, que muestran cómo evolucionan los errores en entrenamiento y validación a medida que aumentan los datos o la complejidad del modelo. Aunque se verán con más detalle en la

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



práctica, estas herramientas permiten mantener un equilibrio entre bias y variance y conseguir modelos más estables y fiables.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Lección 2. Modelos supervisados para clasificación: algoritmos, métricas y usos

Papel de la clasificación, ejemplos de aplicación práctica

La clasificación es un tipo fundamental de problema en aprendizaje automático que se centra en asignar a cada observación una etiqueta o clase específica, basándose en un conjunto de características o variables. Estos modelos aprenden a partir de datos previamente etiquetados, donde cada ejemplo ya tiene asignada su etiqueta correcta. Así, el modelo puede reconocer patrones y relaciones en la información para predecir la etiqueta adecuada en nuevas observaciones que no ha visto antes.

Este proceso es especialmente útil en situaciones donde las decisiones deben automatizarse y los datos son complejos o abundantes, lo que dificulta aplicar reglas fijas o realizar análisis manual. La clasificación se aplica en áreas tan diversas como la detección de fraudes, el diagnóstico médico, la segmentación de clientes, el reconocimiento de imágenes o el análisis de texto. En cada caso, el modelo aprende a distinguir entre diferentes etiquetas y asignar la correcta con la mayor precisión posible.

Aunque un modelo pueda parecer bueno al observar su porcentaje de aciertos, en muchos casos no es suficiente. Dependiendo del problema, ciertos errores pueden tener consecuencias graves, por lo que se utilizan métricas específicas para medir mejor la calidad y seguridad del modelo, como la precisión, sensibilidad o F1-score.

Métricas para evaluar clasificación

Clasificación binaria y multiclase

La clasificación es una tarea en la que el modelo asigna una categoría o clase a cada muestra, basándose en sus características. Puede ser de dos tipos principales: binaria o multiclase. En la clasificación binaria, solo existen dos clases posibles, como "aprobado" o "no aprobado", "spam" o "no spam", "enfermo" o "sano". El modelo decide entre estas dos opciones.

Por otro lado, en la clasificación multiclase se trabaja con más de dos clases, como en un problema de identificar animales entre "gato", "perro" y "conejo". Aquí, el modelo debe escoger entre varias categorías, no solo dos.

Las métricas de evaluación deben adaptarse a cada situación. En binaria, las métricas se calculan en función de los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos, siempre para una clase que se considera positiva. En multiclase, las métricas se amplían para tener en cuenta todas las clases, utilizando enfoques como el macro promedio, que promedia los valores calculados para cada clase, o el micro promedio, que agrupa todos los aciertos y errores para evaluarlos de forma global. Esta adaptación es esencial para tener una visión justa del rendimiento del modelo en todos los casos.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Matriz de confusión para clasificación binaria

La matriz de confusión es una herramienta que permite ver cómo de bien o mal está funcionando un modelo de clasificación. Muestra los casos en los que el modelo ha acertado y los casos en los que se ha equivocado. Es una tabla que cruza las predicciones del modelo con las clases reales.

En el problema de clasificación binaria que vamos a utilizar, la matriz de confusión tiene dos filas y dos columnas. Cada celda representa un grupo de predicciones. La primera fila muestra los casos predichos como "Sano" y la segunda fila los casos predichos como "VIH". La primera columna indica las clases reales "Sano" y la segunda columna las clases reales "VIH". Así se pueden ver los verdaderos positivos (TP), los verdaderos negativos (TN), los falsos positivos (FP) y los falsos negativos (FN).

	Real Sano	Real VIH
Predicho Sano	95 (TP)	20 (FP)
Predicho VIH	5 (FN)	30 (TN)

¿Qué se puede observar?

- La clase positiva es "Sano"
- El número de casos reales de pacientes es 150 ($95 + 5 + 20 + 30$)
- El número de casos reales de personas sanas es 100 ($95 + 5$)
- El número de casos reales de personas con VIH es 50 ($20 + 30$)
- Hay dos clases, "Sano" y "VIH"
- El verdadero positivo o True Positive (TP) es el número que indica los casos en los que el modelo ha clasificado correctamente la clase positiva. En este ejemplo, de 100 personas sanas, ha clasificado correctamente como sanas a 95
- El falso negativo o False Negative (FN) es el número que indica los casos en los que el modelo ha clasificado incorrectamente la clase positiva. En este ejemplo, de 100 personas sanas, ha clasificado incorrectamente como "VIH" a 5
- El falso positivo o False Positive (FP) es el número que indica los casos en los que el modelo ha clasificado incorrectamente la clase no positiva. En este ejemplo, de 50 personas con VIH, ha clasificado incorrectamente como sanas a 20
- El verdadero negativo o True Negative (TN) es el número que indica los casos en los que el modelo ha clasificado correctamente la clase no positiva. En este ejemplo, de 50 personas con VIH, ha clasificado correctamente como "VIH" a 30

¿Qué se puede deducir?

- Por supuesto, 150 observaciones para los datos de validación es insuficiente
- Que los FP son muy peligrosos y siempre hay que tenerlos en cuenta. En primer lugar, porque si un clasificador de 50 casos clasifica incorrectamente casi la mitad, pasa algo. Está mal programado, mal ajustado en los hiperparámetros, no se está usando el algoritmo adecuado o el conjunto de datos previamente no ha sido tratado correctamente

Profundizando más

- Obviando los dos puntos anteriores y considerando que la muestra del conjunto de datos realmente fuese significativa en cuanto al número de observaciones, pero que en porcentaje

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



dé el mismo resultado, y que se haya validado y optimizado el modelo de varias formas y se haya asegurado que los datos no están sesgados pero sigue mostrando porcentajes similares, con un alto número de FP, el clasificador no se podría admitir. La naturaleza de los datos no permite usar un modelo con estos resultados. No se trata de una clasificación sin importancia, las consecuencias de un diagnóstico incorrecto pueden ser fatales

- Los FN indican que a 5 personas se les ha dicho que podrían tener VIH, aunque en realidad no es cierto. Seguramente estas personas entrarían en una segunda fase de pruebas confirmatorias donde finalmente se vería que realmente no tienen la infección, pero habrían pasado un momento de angustia
- Pero los FP están indicando que a 20 personas que realmente tienen VIH, se les dice que no, que están sanas. Qué repercusión tiene este hecho. Imagina, no pasan a una segunda fase de pruebas de confirmación, continúan con su vida habitual, pero la infección sigue avanzando sin que ellas lo sepan. Qué habrá pasado al cabo de un tiempo

Conclusiones

- Siempre hay que hacer un estudio en conjunto y entendiendo la naturaleza de los datos
- Los FP son muy peligrosos y deben tenerse muy en cuenta en la validación

Matriz de confusión para clasificación multiclase

En una clasificación multiclase, aunque la interpretación pueda parecer mucho más compleja que en su equivalente binaria, abordar su lectura no es tan complicado como podría parecer a primera vista. El hecho de que haya más de dos clases significa que no se puede hablar directamente de clases positivas y negativas como en la clasificación binaria. En la práctica, se considera que hay n clasificadores binarios para las n clases, donde para cada clase se toman como verdaderas las predicciones correctas para esa clase y como falsas todas las demás.

Así, en este ejemplo, se analiza cada clase (Gripe, COVID o Alergia) por separado. Para “Gripe”, todos los casos clasificados correctamente como Gripe son considerados verdaderos, mientras que todos los demás casos (predicciones incorrectas) se consideran falsos. El mismo razonamiento se aplica para las otras dos clases. Esto permite calcular métricas como la precisión y la sensibilidad para cada clase individualmente, considerando cada clase como “positiva” y todas las demás como “negativas” en ese análisis parcial.

	Real Gripe	Real COVID	Real Alergia
Predicho Gripe	40 (T)	5 (F)	10 (F)
Predicho COVID	3 (F)	35 (T)	2 (F)
Predicho Alergia	7 (F)	10 (F)	30 (T)

¿Qué se puede observar?

- La matriz tiene tres clases: Gripe, COVID y Alergia
- El número total de casos reales es 142 ($40 + 5 + 10 + 3 + 35 + 2 + 7 + 10 + 30$)
- El número de casos reales de Gripe es 55 ($40 + 5 + 10$)
- El número de casos reales de COVID es 50 ($3 + 35 + 12$)
- El número de casos reales de Alergia es 37 ($7 + 10 + 30$)

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- No hay clases positivas y negativas como en binaria, pero cada clase se analiza como positiva en su análisis parcial
- El verdadero positivo (TP) para Gripe es el número de casos reales de Gripe clasificados correctamente como Gripe, en este ejemplo son 40
- El falso negativo (FN) para Gripe es el número de casos reales de Gripe que el modelo ha clasificado incorrectamente como COVID o Alergia, en este ejemplo son $5 + 10 = 15$
- El falso positivo (FP) para Gripe son los casos de COVID o Alergia clasificados erróneamente como Gripe, en este ejemplo son $3 + 7 = 10$
- El verdadero negativo (TN) para Gripe son todos los casos que no son de Gripe y que el modelo no ha clasificado como Gripe, en este ejemplo son $35 + 2 + 10 + 30 = 77$
- Lo mismo se repite para cada clase en la matriz, considerando cada clase como “positiva” y todas las demás como “negativas” en su análisis parcial

Exactitud (Accuracy)

La exactitud mide el porcentaje de predicciones correctas que ha hecho el modelo en relación al total de casos.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) = (95 + 30) / (95 + 30 + 20 + 5) = 125 / 150 \approx 0.83$$

Precisión (Precision)

La precisión indica el porcentaje de predicciones positivas que realmente son positivas.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 95 / (95 + 20) = 95 / 115 \approx 0.83$$

Sensibilidad (Recall)

La sensibilidad o recall mide el porcentaje de casos positivos reales que el modelo ha identificado correctamente.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 95 / (95 + 5) = 95 / 100 = 0.95$$

Especificidad

La especificidad mide el porcentaje de casos negativos reales que el modelo ha identificado correctamente.

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP}) = 30 / (30 + 20) = 30 / 50 = 0.6$$

F1-Score

El F1-score es la media armónica entre precisión y sensibilidad, útil cuando hay un desequilibrio entre clases.

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 * (0.83 * 0.95) / (0.83 + 0.95) \approx 0.89$$

Índice Kappa

El índice Kappa mide la concordancia entre las predicciones y las observaciones, ajustada por el azar
 $Kappa = (Accuracy - Azar) / (1 - Azar) = (0.83 - 0.62) / (1 - 0.62) = 0.21 / 0.38 \approx 0.55$

Donde el acuerdo por azar (Azar) se calcula así:

$$Azar = ((TP + FP) * (TP + FN) + (TN + FN) * (TN + FP)) / (Total^2)$$

$$Azar = ((115 * 100) + (50 * 50)) / (150 * 150) = (11500 + 2500) / 22500 = 14000 / 22500 \approx 0.62$$

Valor predictivo positivo (PPV)

El valor predictivo positivo o PPV indica la probabilidad de que un caso clasificado como positivo realmente lo sea.

$$PPV = TP / (TP + FP) = 95 / (95 + 20) = 95 / 115 \approx 0.83$$

Valor predictivo negativo (NPV)

El valor predictivo negativo o NPV indica la probabilidad de que un caso clasificado como negativo realmente lo sea.

$$NPV = TN / (TN + FN) = 30 / (30 + 5) = 30 / 35 \approx 0.86$$

Curva ROC y Área bajo la curva (AUC)

La curva ROC (Receiver Operating Characteristic) es un gráfico que muestra la relación entre la tasa de verdaderos positivos (sensibilidad) y la tasa de falsos positivos (1 - especificidad) para diferentes umbrales de decisión. Ayuda a visualizar cómo cambia el rendimiento del modelo al variar ese umbral.

Para este ejemplo con un único umbral no se puede trazar toda la curva a mano, pero sí se puede ver el punto que representaría la combinación de sensibilidad y 1 - especificidad:

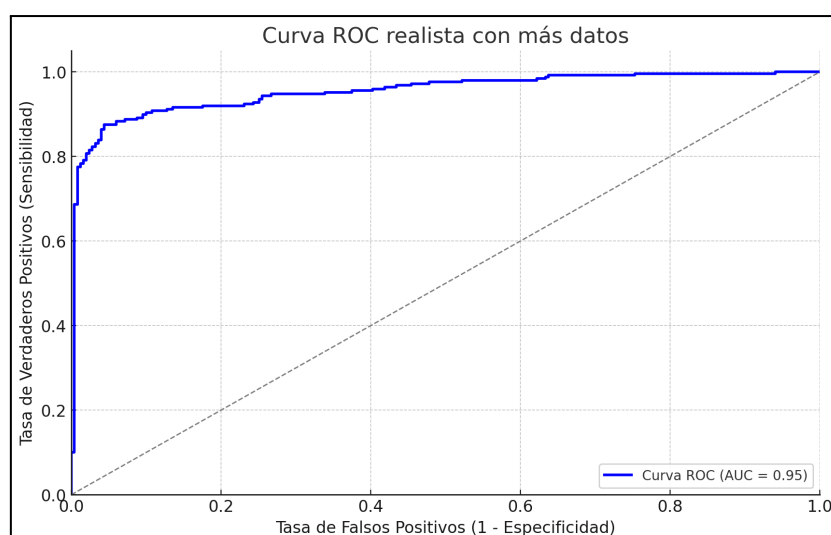
- Sensibilidad: 0.95
- 1 - especificidad: $1 - 0.6 = 0.4$

El AUC (Area Under the Curve) mide el área total bajo la curva ROC y refleja la capacidad general del modelo para discriminar entre clases. Un AUC de 1 indica discriminación perfecta, mientras que un AUC de 0.5 indica rendimiento aleatorio.

En este ejemplo no se puede calcular manualmente el AUC exacto sin la curva completa (necesitamos todos los puntos intermedios generados por varios umbrales).

En la siguiente imagen, imaginemos que el modelo se está utilizando para diagnosticar una enfermedad respiratoria como la COVID-19, con la intención de diferenciar entre personas sanas y personas infectadas.





La curva ROC es un gráfico que muestra qué tan bien funciona este modelo para separar esas dos clases. En el eje horizontal aparece la tasa de falsos positivos, es decir, cuándo el modelo se equivoca y dice que alguien está enfermo cuando no es así. En el eje vertical está la tasa de verdaderos positivos, que indica cuántas personas realmente enfermas el modelo detecta correctamente.

Si la curva sube rápidamente y se acerca a la esquina superior izquierda, eso significa que el modelo es muy preciso, detecta bien a los enfermos y comete pocos errores. Pero si la curva está cerca de la diagonal gris, indica que el modelo no es mejor que adivinar al azar. El área bajo la curva (AUC) mide qué tan buena es esta separación, donde un valor de 1 significa que el modelo es perfecto y 0.5 que no aporta nada.

En la curva que vemos, la línea azul sube rápido al principio y luego se aplan, lo que indica que el modelo detecta la mayoría de casos reales sin aumentar demasiado los falsos positivos. El AUC cercano a 1 confirma que el modelo es fiable y se puede usar para apoyar decisiones médicas.

Elección de métricas según el contexto del problema

La evaluación de un modelo de clasificación depende mucho del contexto y de las consecuencias que tengan los errores que pueda cometer. No todas las métricas son igual de importantes para todos los problemas, porque cada una mide distintos tipos de aciertos o fallos.

Por ejemplo, en algunos casos es muy peligroso que un modelo clasifique como sano a alguien que en realidad está enfermo, como en el caso del VIH. Aquí los falsos positivos (FP) son críticos, y por eso es fundamental prestar atención a la precisión, que mide qué porcentaje de las personas que el modelo dice que están sanas realmente lo están.

En cambio, en enfermedades muy contagiosas o graves, puede ser más importante no dejar escapar ningún caso real, aunque eso implique dar algunas falsas alarmas. En estos casos se prioriza la sensibilidad o recall, que mide qué porcentaje de personas enfermas el modelo detecta correctamente.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Además, cuando las clases están desbalanceadas (por ejemplo, muy pocos enfermos y muchos sanos), la exactitud o accuracy puede dar una falsa sensación de buen rendimiento porque el modelo puede acertar mucho simplemente prediciendo la clase mayoritaria. En estas situaciones, métricas como el F1-score, que equilibran precisión y sensibilidad, ayudan a entender mejor el verdadero rendimiento del modelo.

Por eso, elegir qué métrica usar para evaluar un modelo no es cuestión de usar siempre la misma, sino de pensar cuál refleja mejor los riesgos y prioridades del problema concreto, para tomar decisiones más acertadas.

Detección del sobreajuste en clasificación y análisis visual

Un modelo de clasificación puede mostrar una precisión muy alta tanto en entrenamiento como en validación, y aun así no ser fiable si no ha aprendido a generalizar. Este tipo de modelos funcionan correctamente con los datos que ya conocen, pero pueden fallar al enfrentarse a nuevos casos reales o a datos ligeramente distintos. En otras palabras, han aprendido demasiado bien los detalles del entrenamiento, incluyendo ruido o patrones poco representativos.

Para asegurarse de que un modelo no está sobreajustado, no basta con observar su precisión en test o en validación cruzada. Es necesario hacer comprobaciones adicionales y analizar cómo se comporta en diferentes situaciones. A continuación, veremos algunas técnicas comunes para detectarlo.

Validación cruzada

La validación cruzada es una técnica que permite evaluar la capacidad de un modelo para generalizar a datos no vistos. En lugar de entrenar y probar el modelo una sola vez, se repite el proceso varias veces utilizando diferentes particiones del conjunto de datos.

Como se muestra en la imagen, el conjunto se divide en varios subconjuntos llamados *folds*. En cada iteración, uno de ellos se utiliza como conjunto de prueba (*Test*, en color distinto) y los demás como conjunto de entrenamiento (*Train*). Este proceso se repite tantas veces como folds haya, de modo que cada partición actúa una vez como test. En la figura pueden verse claramente las cinco iteraciones, cada una usando un fold diferente como test.

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Test	Train	Train	Train	Train	Iteración 1
Train	Test	Train	Train	Train	Iteración 2
Train	Train	Test	Train	Train	Iteración 3
Train	Train	Train	Test	Train	Iteración 4
Train	Train	Train	Train	Test	Iteración 5

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Al finalizar todas las iteraciones, se calcula una métrica de evaluación en cada una de ellas —como el error cuadrático medio (EQM), el error absoluto medio (EAM) o el coeficiente de determinación R^2 — y se obtiene su media. Este valor promedio resume el rendimiento general del modelo, mostrando cómo se comporta ante distintas particiones. También puede calcularse la desviación estándar, que indica si el modelo es estable: una desviación baja sugiere que mantiene un rendimiento constante, mientras que una alta puede señalar sensibilidad a ciertos subconjuntos.

Este procedimiento tiene dos ventajas clave. Por un lado, proporciona una medida más robusta del rendimiento al evaluar sobre múltiples subconjuntos. Por otro, ayuda a detectar si el modelo depende demasiado de ciertos datos, lo que puede indicar sobreajuste o inestabilidad si los resultados varían mucho entre iteraciones.

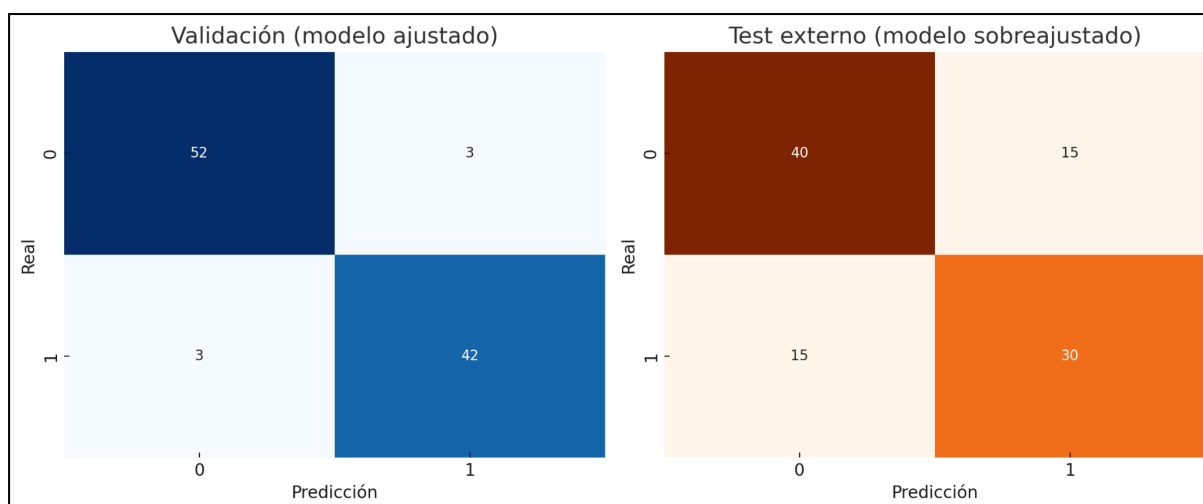
La validación cruzada es especialmente útil en regresión, donde es frecuente encontrar datos desbalanceados o con valores atípicos. Además, resulta muy eficaz para ajustar hiperparámetros, ya que permite evaluar el comportamiento del modelo en distintos escenarios sin necesidad de reservar un conjunto externo de validación.

Comparación entre entrenamiento y test

Una señal de sobreajuste es que el modelo tenga una precisión casi perfecta sobre los datos de entrenamiento, pero un rendimiento claramente inferior en test. Esto indica que el modelo está memorizando los datos en lugar de aprender patrones generales que funcionen con nuevos ejemplos.

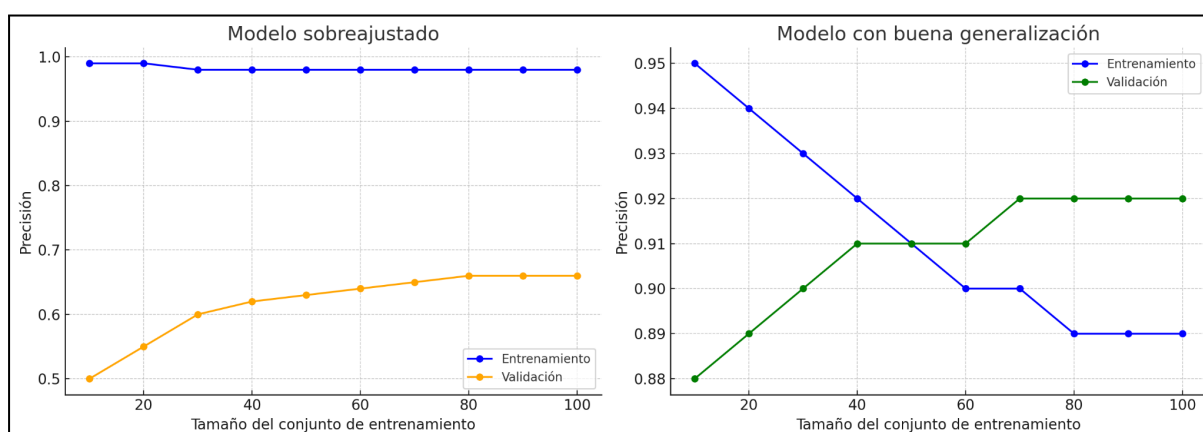
En la siguiente imagen se puede observar a la izquierda se muestra la matriz del conjunto de validación. El modelo comete pocos errores: clasifica correctamente la mayoría de los positivos (42 de 45) y negativos (52 de 55). Esto indica un buen rendimiento durante la validación.

A la derecha se muestra la matriz de test externo. Aquí el rendimiento cae: el modelo acierta solo 30 de los 45 positivos y 40 de los 55 negativos, con más errores de clasificación. Esta diferencia entre validación y test sugiere que el modelo está sobreajustado a los datos de validación y no generaliza bien con nuevos casos.



Curva de aprendizaje

Las curvas de aprendizaje permiten observar cómo evoluciona el rendimiento del modelo a medida que aumentamos el número de muestras disponibles. Si el modelo mejora en validación conforme se añaden más datos y la distancia entre entrenamiento y validación se reduce, es buena señal. Pero si la precisión en validación se estanca o empeora mientras la de entrenamiento sigue aumentando, probablemente el modelo está sobreajustando.



Validación con datos externos o nuevas observaciones

Otra forma de evaluar si un modelo generaliza bien es probarlo con un conjunto de datos externo o con observaciones nuevas que simulan situaciones reales o incluso casos extremos. Esto permite ver si el modelo responde correctamente ante inputs distintos a los vistos durante el entrenamiento. Si su rendimiento sigue siendo bueno, hay más garantías de que el modelo no está sobreajustado.

Por tanto, elegir un buen modelo no se basa únicamente en una métrica concreta, sino en un análisis más amplio que combine diferentes pruebas y perspectivas. A lo largo del apartado se han presentado los principales indicadores para detectar sobreajuste y evaluar la capacidad de

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



generalización. Estos conceptos se entenderán con mayor claridad al aplicarlos en un ejemplo práctico a continuación.

Algoritmos de clasificación

Introducción a los algoritmos de clasificación

La clasificación es una tarea fundamental dentro del aprendizaje supervisado, cuyo objetivo es asignar a cada dato una etiqueta o clase entre un conjunto definido de opciones. A diferencia de la regresión, que predice valores continuos, la clasificación trabaja con variables discretas y categorías, como “aprobado” o “suspense” en educación, “fraude” o “transacción legítima” en banca, o “canción de rock” o “canción de pop” en análisis de música.

Existen múltiples algoritmos diseñados para resolver problemas de clasificación, cada uno con su propia lógica y mecanismos para aprender de los datos. Algunos se basan en principios probabilísticos, otros en la proximidad o distancia entre datos, y otros en encontrar límites o reglas que separan las clases.

Conocer estos algoritmos y entender sus ventajas, limitaciones y casos de uso es esencial para seleccionar el modelo más adecuado a cada problema. La elección correcta influye directamente en la eficacia del modelo, su capacidad para generalizar y su utilidad práctica.

Naive Bayes fundamentos probabilísticos y supuestos

Idea principal y ejemplos de uso

Naive Bayes es un algoritmo de clasificación basado en la probabilidad. Parte del teorema de Bayes y asume que las características que describen una observación son independientes entre sí dentro de cada clase, aunque en la práctica esta suposición rara vez se cumple totalmente. Aun así, el modelo resulta muy eficaz porque simplifica los cálculos y funciona bien cuando las relaciones entre las variables no son muy complejas.

Se utiliza frecuentemente en clasificación de textos (como detección de spam o análisis de sentimientos), donde las variables suelen ser palabras individuales y la independencia entre ellas puede aproximarse razonablemente. También es útil cuando los datos están claramente separados por patrones simples o cuando se necesita una solución rápida y eficiente. Es especialmente adecuado para datos categóricos o discretos y funciona mejor cuando las distribuciones dentro de cada clase no están demasiado solapadas. En situaciones con muchas variables pero pocas observaciones, puede superar a modelos más complejos.

Cómo funciona

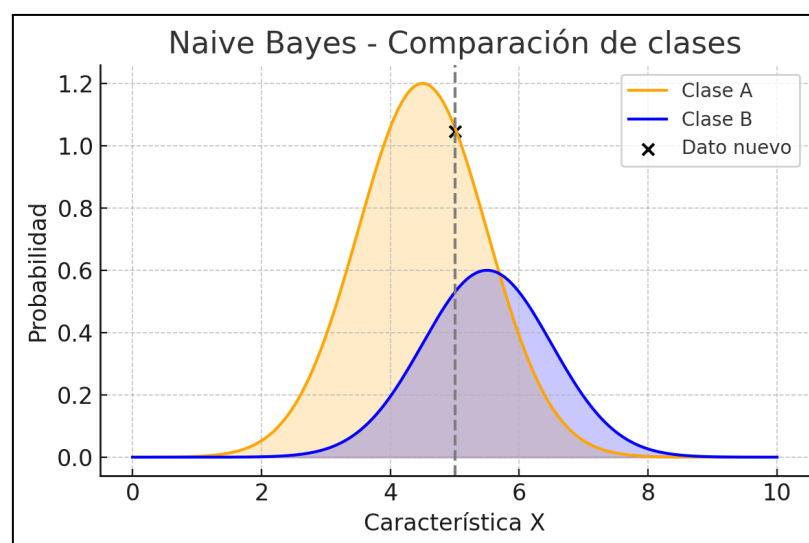
Para cada nueva observación, el algoritmo calcula la probabilidad de pertenencia a cada clase basándose en la frecuencia de las características en los datos de entrenamiento. Multiplica las probabilidades individuales de cada característica, considerando la clase, y escoge la que tenga el valor más alto.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Por ejemplo, en un sistema que clasifica comentarios de usuarios entre positivos y negativos, si el texto contiene palabras como “horrible”, “malo” y “decepción”, el modelo comprueba qué probabilidad tiene cada una de esas palabras de aparecer en comentarios negativos o positivos y, al combinar la información, asigna la clase más probable.

O, en esta imagen se representan dos curvas de probabilidad, una para cada clase. La curva naranja corresponde a la clase A y la azul a la clase B. Ambas muestran cómo se distribuyen los valores de una característica concreta dentro de cada clase. El eje horizontal representa los posibles valores de esa característica, y el eje vertical indica la probabilidad de observar cada valor dentro de cada clase. En el centro del gráfico aparece una línea discontinua con un punto negro que representa un nuevo dato que queremos clasificar. Para tomar la decisión, el algoritmo compara la altura de ambas curvas en ese punto. Como la curva naranja es más alta que la azul, el modelo considera que ese valor es más probable bajo la clase A y clasifica el nuevo dato como perteneciente a esa clase. Esta es la base del funcionamiento de Naive Bayes: asigna la clase con mayor probabilidad condicional en función del valor observado.



Puntos clave del algoritmo

- Parte de un modelo probabilístico que se apoya en el teorema de Bayes
- Hace el supuesto fuerte de independencia entre características
- Necesita muy pocos recursos computacionales y entrena en segundos
- Ofrece buenos resultados cuando hay muchas variables discretas
- Suele ser el punto de partida en tareas de clasificación con texto

Cuándo funciona bien y cuándo no

Rinde especialmente bien en problemas donde los datos son abundantes pero sencillos, como texto categorizado o formularios con variables cerradas. También funciona bien cuando los patrones de cada clase están bien diferenciados, aunque las características no sean del todo independientes.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



En cambio, pierde precisión cuando las variables están fuertemente correlacionadas o cuando se trata de problemas donde las relaciones entre variables son complejas y no lineales. Tampoco es la mejor opción cuando hay que capturar interacciones entre variables o tomar decisiones finas entre clases muy similares.

Particularidades del algoritmo

Un aspecto técnico relevante en los clasificadores Naive Bayes es el problema de la probabilidad cero. Este ocurre cuando una característica con cierto valor no aparece en el conjunto de entrenamiento para una clase concreta. Como el modelo calcula la probabilidad conjunta multiplicando las probabilidades de cada característica, una sola probabilidad igual a cero anula toda la predicción. Para evitarlo, se utiliza el suavizado de Laplace, que consiste en sumar una pequeña constante (habitualmente 1) a todos los conteos. Esto garantiza que ninguna probabilidad condicional sea cero, haciendo el modelo más robusto frente a combinaciones nuevas de valores.

Naive Bayes también destaca por su capacidad para trabajar con conjuntos de datos de alta dimensionalidad, especialmente cuando las variables son discretas o categóricas, como ocurre en clasificación de texto, análisis de sentimientos o filtrado de spam. No necesita reducción de dimensionalidad previa, lo que lo hace eficiente incluso con miles de características.

Se distinguen varios tipos de clasificadores Naive Bayes según el tipo de datos que se quiera modelar:

- Naive Bayes Gaussiano: asume que las características siguen una distribución normal. Se aplica en problemas con datos continuos y cuantitativos, donde cada clase tiene su propia media y desviación estándar para cada atributo.
- Naive Bayes Multinomial: adecuado para datos categóricos o conteo de frecuencias, como el número de veces que aparece una palabra en un documento. Se usa habitualmente en clasificación de texto. En casos binarios (presencia/ausencia), puede derivarse en una distribución binomial o, más habitualmente, en la distribución de Bernoulli, pensada para variables booleanas.
- Naive Bayes Ordinal: útil cuando las características categóricas tienen un orden natural. En lugar de tratar las categorías como independientes, este modelo aprovecha la jerarquía implícita, y estima la probabilidad de que la clase esté en una categoría concreta o inferior.

K-Nearest Neighbors distancia y maldición de la dimensionalidad

Idea principal y ejemplos de uso

K-Nearest Neighbors (KNN) es un algoritmo de clasificación que decide la clase de una observación nueva en función de las clases de sus vecinos más cercanos. La idea es sencilla: si algo se parece a ciertos elementos conocidos, probablemente pertenezca a la misma categoría.

KNN es muy útil en problemas donde no se dispone de un modelo explícito pero sí se tiene una buena cantidad de ejemplos bien etiquetados. Se aplica con frecuencia en reconocimiento de imágenes, detección de anomalías o clasificación de clientes. Funciona especialmente bien cuando las clases están bien separadas en el espacio de características y la relación entre los datos puede entenderse mediante distancias.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Está pensado para datos continuos o normalizados, ya que cualquier variable dominante puede distorsionar el cálculo de distancias. También se adapta mejor a distribuciones con agrupaciones claras y pocas dimensiones, ya que la efectividad se reduce drásticamente cuando aumenta la cantidad de variables irrelevantes o se trabaja en espacios de alta dimensión.

Cómo funciona

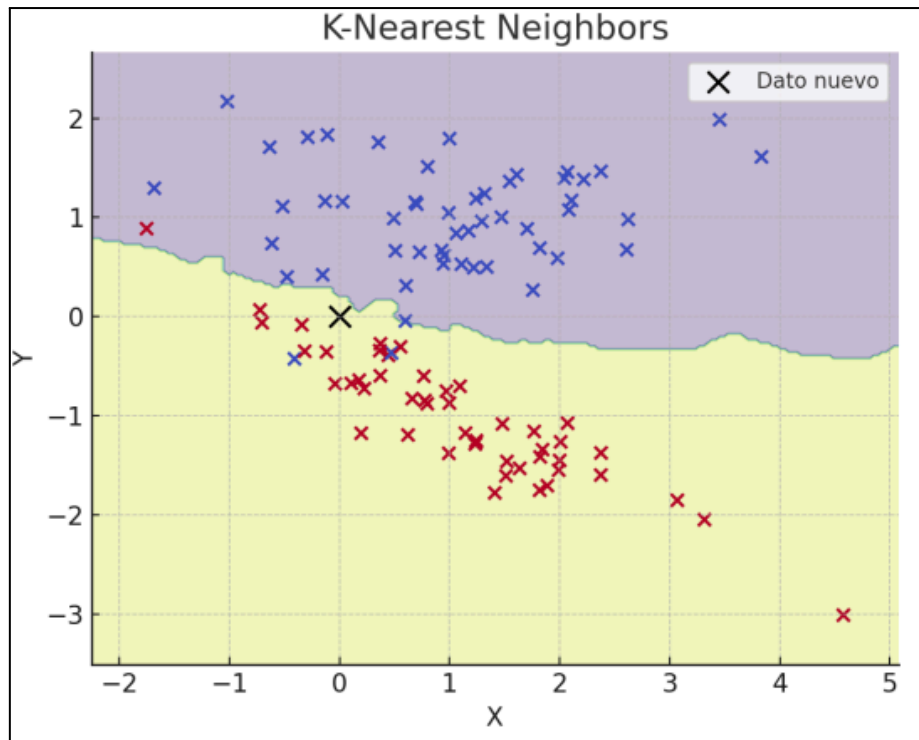
Cuando se quiere clasificar una nueva instancia, el algoritmo calcula la distancia entre ese punto y todos los puntos del conjunto de entrenamiento. Luego selecciona los k más cercanos y asigna la clase mayoritaria entre ellos. Cuanto menor sea la distancia, mayor será la influencia de ese vecino en la decisión.

Por ejemplo, si se quiere predecir el tipo de fruta según color y tamaño, y las frutas más cercanas en esos valores son tres manzanas y una pera, la nueva fruta se clasificará como manzana.

O, en esta imagen se representa cómo clasifica un modelo K-Nearest Neighbors en un espacio bidimensional. Los puntos del conjunto de entrenamiento están marcados en azul y rojo, según su clase. El fondo está dividido en dos regiones: la lila corresponde a la clase azul y la amarilla a la clase roja. Estas regiones indican cómo se clasifica cualquier nuevo dato según la mayoría de sus vecinos más cercanos.

En el centro de la imagen aparece un punto negro que representa un dato nuevo. Aunque a simple vista parece estar próximo a varios puntos azules, cae dentro de la región amarilla. Eso significa que, al aplicar el algoritmo, la mayoría de los k vecinos más cercanos según la distancia definida pertenecen a la clase roja, por lo que el modelo lo clasifica como tal.

Este caso es especialmente representativo porque pone de manifiesto una de las particularidades del algoritmo: la decisión depende de qué puntos se consideran “más cercanos”, y eso a su vez depende de la métrica de distancia utilizada. La más común es la euclídea, pero pueden usarse otras como Manhattan o Minkowski, que pueden cambiar la clasificación final si la distribución de los datos es diferente.



Puntos clave del algoritmo

- No requiere entrenamiento previo, solo guarda los datos
- Depende directamente de la elección del número de vecinos (k)
- Requiere una métrica de distancia adecuada (habitualmente Euclídea)
- Sensible a la escala de los datos: el preprocesado es fundamental
- Cuanto mayor es k , más estable la predicción, pero menos precisa en los bordes

Cuándo funciona bien y cuándo no

Funciona especialmente bien en problemas donde las clases están bien separadas en el espacio de características y donde la distancia es un indicador fiable de similitud. También se comporta bien en conjuntos pequeños o moderados y cuando se pueden normalizar las variables.

En cambio, sufre mucho en espacios con muchas dimensiones (maldición de la dimensionalidad), cuando hay ruido en los datos o cuando las clases no están claramente delimitadas. También puede ser muy lento si el conjunto de datos es grande y no se optimiza con estructuras como árboles de búsqueda.

Particularidades del algoritmo

Una de las decisiones más importantes al usar KNN es cuántos vecinos tener en cuenta, es decir, el valor de k . Si se elige un k pequeño, el modelo tiende a ajustarse demasiado a los datos y se vuelve muy sensible al ruido. Por ejemplo, si k es 1, basta que un solo dato esté mal etiquetado para que afecte la predicción. En cambio, si k es muy grande, el modelo puede suavizar demasiado la decisión y considerar vecinos de clases distintas, lo que también reduce la precisión. Por eso, la elección de k

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



se suele hacer probando diferentes valores con validación cruzada. Se observa el rendimiento para cada k y se escoge el que consigue mejores resultados en validación. También es frecuente usar valores impares para evitar empates.

Además del valor de k , el tipo de distancia que se usa es clave. La distancia euclídea es la más común, pero en algunos casos puede ser más útil usar otras métricas como la Manhattan o la de Minkowski, según la naturaleza de los datos. El tipo de distancia también puede influir en la elección óptima de k , ya que una métrica puede separar mejor las clases que otra. Por eso, la búsqueda del mejor valor de k debe ir siempre acompañada de pruebas con distintas distancias. También es necesario escalar las variables, ya que si una característica tiene un rango mucho más amplio que las otras, dominará el cálculo de distancias y distorsionará los resultados.

Por otro lado, KNN tiene un problema conocido como la maldición de la dimensionalidad. Cuando el número de características es muy alto, todos los puntos acaban estando casi igual de lejos entre sí, y el concepto de "vecino más cercano" deja de tener sentido. Esto reduce la capacidad del algoritmo para tomar decisiones útiles, por lo que conviene aplicar reducción de dimensiones o seleccionar solo las variables más relevantes antes de usar KNN.

Finalmente, este algoritmo no aprende un modelo explícito, sino que memoriza todos los datos y decide en el momento de la predicción. Esto le da flexibilidad, pero también hace que sea poco interpretable y que requiera más memoria y tiempo cuando hay muchos datos.

Regresión logística decisión lineal y regularización

Idea principal y ejemplos de uso

La regresión logística es un algoritmo de clasificación que calcula la probabilidad de que una observación pertenezca a una clase u otra, a partir de una combinación lineal de sus características. Aunque lleva el nombre de regresión, se utiliza para tareas de clasificación binaria.

Su funcionamiento es simple pero potente y es muy utilizado en contextos donde se desea entender el peso de cada variable sobre el resultado. Se aplica en campos como la detección de enfermedades, el análisis de riesgo financiero, la predicción de abandono de clientes o cualquier situación con una respuesta sí/no. Funciona bien en entornos donde hay muchos datos pero los patrones son simples o lineales.

Cómo funciona

El modelo combina de forma lineal todas las variables de entrada, y a ese resultado le aplica una función sigmoide que transforma cualquier número real en un valor entre 0 y 1. Ese valor representa la probabilidad de que el dato pertenezca a la clase positiva.

La decisión final se toma comparando esa probabilidad con un umbral, normalmente 0.5. Si la probabilidad es mayor, se asigna la clase 1. Si es menor, se asigna la clase 0.

El entrenamiento del modelo consiste en encontrar los valores adecuados para los coeficientes que acompañan a cada variable. El objetivo es que, al aplicar la combinación lineal y pasarla por la función sigmoide, las probabilidades resultantes estén lo más cerca posible de las clases reales.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Para medir qué tan buena es una predicción, se utiliza una función llamada log-loss, que penaliza mucho más los errores de predicción cuando el modelo está muy seguro y se equivoca. Por ejemplo, si el modelo predice una probabilidad de 0.99 pero el dato pertenece a la clase contraria, el castigo será mayor.

El proceso de entrenamiento ajusta los coeficientes poco a poco, reduciendo esa penalización global hasta que las probabilidades asignadas por el modelo se alinean lo mejor posible con los datos reales.

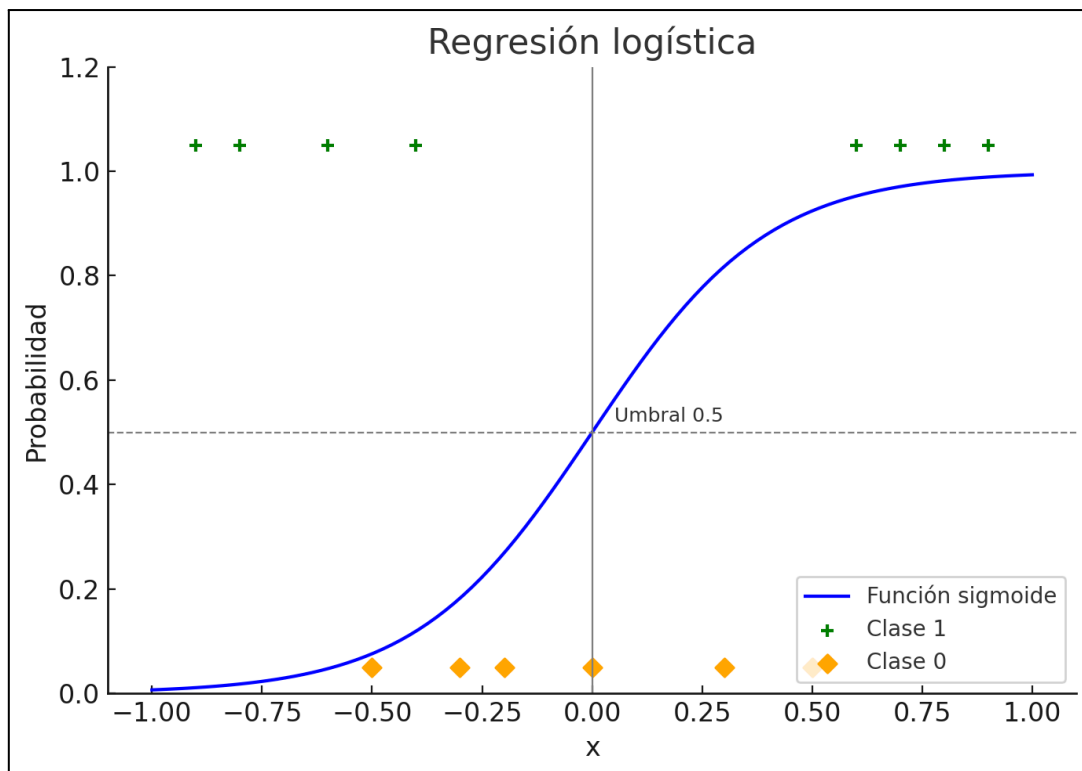
En esta imagen se muestra cómo funciona la regresión logística para un problema de clasificación binaria.

La línea azul representa la función sigmoide, que transforma los valores de entrada (x) en una probabilidad entre 0 y 1. Esta probabilidad indica qué tan probable es que un dato pertenezca a la clase positiva (clase 1).

Los puntos naranjas en la parte inferior representan ejemplos de la clase 0, y los puntos verdes con signo + en la parte superior representan ejemplos de la clase 1. Estos puntos se sitúan visualmente por encima o por debajo de la curva para indicar su clase, aunque en realidad todos corresponden a un valor binario (0 o 1).

La línea gris discontinua marca el umbral de decisión, que normalmente se fija en 0.5. Si la probabilidad predicha por la curva es mayor a 0.5, el modelo asigna la clase 1. Si es menor o igual a 0.5, asigna la clase 0.

Esto significa que los datos a la izquierda de $x = 0$ suelen clasificarse como clase 0, y los que están a la derecha como clase 1. El modelo ajusta la curva durante el entrenamiento para que la mayor parte de los puntos queden correctamente clasificados respecto al umbral.



Puntos clave del algoritmo

- Genera una frontera de decisión lineal
- No transforma los datos, solo calcula una probabilidad
- Permite interpretar fácilmente la influencia de cada variable
- Requiere que las variables estén escaladas si tienen rangos distintos
- Se puede regularizar para evitar sobreajuste, especialmente cuando hay muchas variables

Cuándo funciona bien y cuándo no

Da buenos resultados cuando las clases se pueden separar de forma aproximada con una línea recta o un plano, y cuando no hay relaciones demasiado complejas entre variables. También funciona bien si se necesita rapidez, interpretabilidad o si el conjunto de datos no es muy grande.

Sin embargo, pierde eficacia si la separación entre clases es no lineal o si hay muchas interacciones entre variables que no pueden capturarse con una combinación lineal. En esos casos, otros modelos más flexibles suelen rendir mejor.

Particularidades del algoritmo

Uno de los aspectos más destacados es que la regresión logística permite aplicar regularización, una técnica clave para evitar que el modelo se adapte demasiado a los datos de entrenamiento. Cuando hay muchas variables o poca información, los coeficientes del modelo pueden volverse muy grandes

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



e inestables. La regularización añade un término extra a la función de coste que penaliza los valores extremos, obligando al modelo a mantener los coeficientes controlados.

Existen principalmente dos tipos de regularización:

- L2 o Ridge, que penaliza la suma de los cuadrados de los coeficientes. No elimina variables, pero las reduce
- L1 o Lasso, que penaliza la suma de los valores absolutos de los coeficientes. Esto permite que algunos se reduzcan a cero, actuando como una forma de selección automática de variables

También se pueden combinar ambas técnicas en lo que se llama regularización elástica.

Además, el modelo permite ajustar el umbral de decisión según el problema. Por ejemplo, en contextos médicos o de seguridad, es preferible detectar el máximo número posible de casos positivos aunque aumenten los falsos positivos. En esos casos, se puede bajar el umbral por debajo de 0.5 para que el modelo sea más sensible. Esto convierte a la regresión logística en una herramienta no solo eficaz sino también flexible y fácil de adaptar a distintos tipos de decisiones.

Support Vector Machines margen máximo y kernels

Idea principal y ejemplos de uso

Support Vector Machines (SVM) es un algoritmo de clasificación que busca separar las clases con la máxima claridad posible. A diferencia de otros métodos que se centran en minimizar errores, SVM prioriza encontrar la frontera que deje el mayor margen posible entre las clases, para ser más robusto frente al ruido.

Se utiliza en reconocimiento de imágenes, clasificación de texto, bioinformática y problemas donde las clases están bien estructuradas o se requiere alta precisión. Es muy eficaz con muchas características y pocos datos, y se adapta bien a contextos en los que las relaciones no son lineales.

Cómo funciona

El algoritmo SVM busca una línea o superficie que separe las clases y maximice la distancia con respecto a los puntos más cercanos, conocidos como vectores soporte. Esta frontera se llama hiperplano de decisión, y solo los puntos cercanos al límite influyen en su construcción.

Sin embargo, a veces no es posible trazar una línea recta que separe bien las clases en el espacio original. En estos casos, se dice que las clases no son linealmente separables. Para resolver este tipo de situaciones, SVM utiliza una técnica llamada kernel, que permite transformar los datos a un espacio de mayor dimensión donde sí es posible separarlos con un hiperplano.

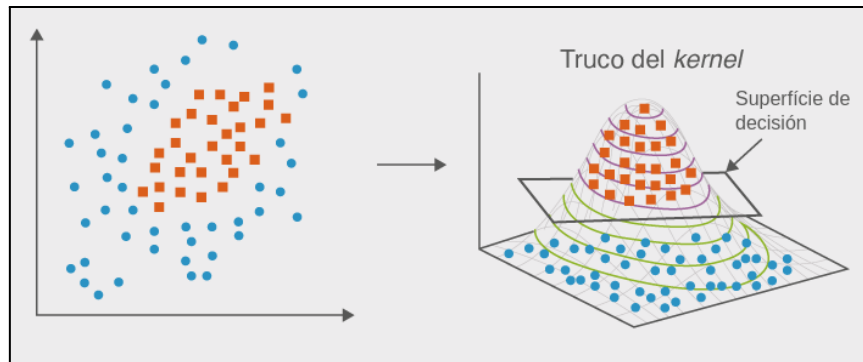
Esta transformación no se realiza de forma explícita. Gracias al kernel, el modelo actúa como si los datos estuvieran proyectados en ese nuevo espacio, sin necesidad de calcular las nuevas coordenadas directamente. Allí, la separación entre clases se vuelve más sencilla.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



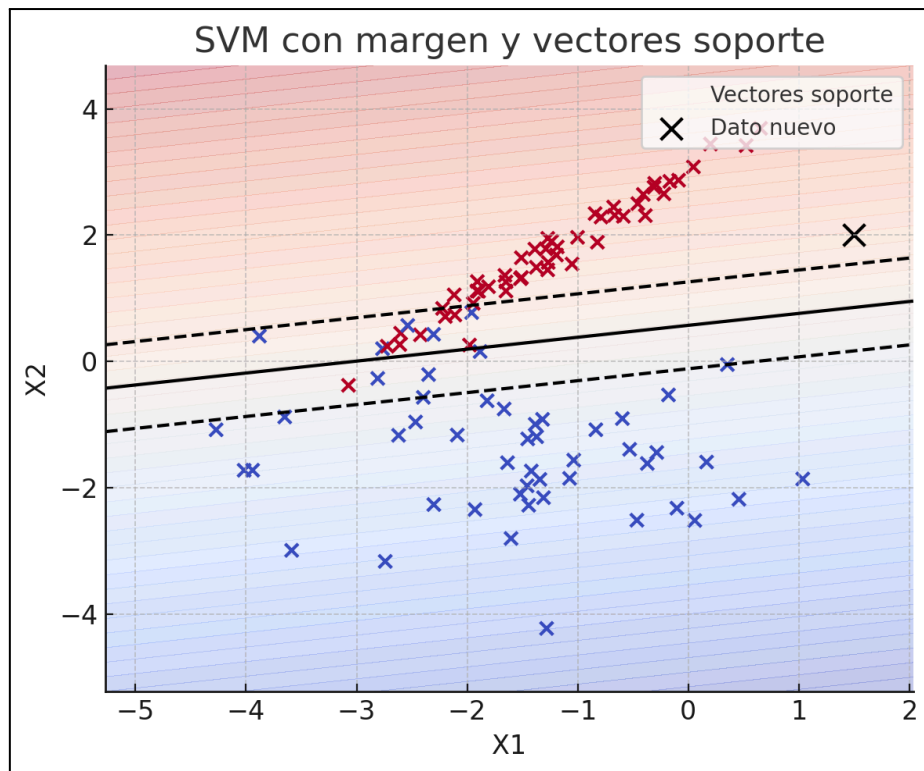
En la imagen, a la izquierda se ven los datos en dos dimensiones, donde no existe una línea recta que los separe correctamente. A la derecha, tras aplicar la transformación del kernel, los datos se representan en un espacio tridimensional donde sí es posible dividir las clases mediante una superficie de decisión plana.

Esta capacidad de proyectar los datos a una dimensión superior permite que SVM pueda resolver problemas complejos que no se pueden abordar con modelos lineales simples.



En la siguiente imagen los puntos azules y rojos representan dos clases diferentes. La línea central es el hiperplano que separa ambas clases con la mayor distancia posible. Las líneas discontinuas marcan los márgenes, que indican hasta dónde llega el espacio seguro de cada clase. Los puntos que tocan estos márgenes se llaman vectores soporte y son los únicos que influyen en la construcción de esa frontera.

El punto nuevo, marcado con una X, cae del lado de los puntos rojos, por eso el modelo lo clasifica dentro de esa clase. Aunque esté dentro del margen, eso no impide la clasificación. El margen no representa una zona de duda, sino una zona de seguridad que el modelo intenta maximizar durante el entrenamiento para ser más robusto frente a errores o ruido en los datos.



Puntos clave del algoritmo

- Maximiza el margen entre clases para mejorar la generalización
- Solo utiliza los vectores soporte para construir la frontera
- Permite transformar el espacio mediante funciones kernel
- Incluye parámetros que controlan el equilibrio entre precisión y generalización
- Puede utilizarse para clasificación binaria o multiclase

Cuándo funciona bien y cuándo no

SVM funciona especialmente bien cuando hay una separación clara entre clases o cuando se pueden representar bien con una transformación. También es muy útil cuando hay pocas observaciones pero muchas características, ya que el modelo solo se apoya en los casos más relevantes.

Sin embargo, pierde precisión cuando hay mucho solapamiento o ruido, ya que puede fijarse demasiado en casos atípicos. También puede ser lento con conjuntos de datos grandes, porque necesita comparar todos los puntos. Además, no es fácil de interpretar y suele requerir bastante ajuste de parámetros.

Particularidades del algoritmo

SVM es un modelo potente y versátil, pero requiere entender bien cómo funcionan sus parámetros y transformaciones.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Una de las claves del algoritmo es el uso de kernels. Un kernel es una función que calcula el producto escalar entre puntos transformados a un espacio de dimensión más alta, sin tener que calcular esa transformación directamente. Gracias a eso, SVM puede resolver problemas donde las clases no se pueden separar con una línea recta en el espacio original.

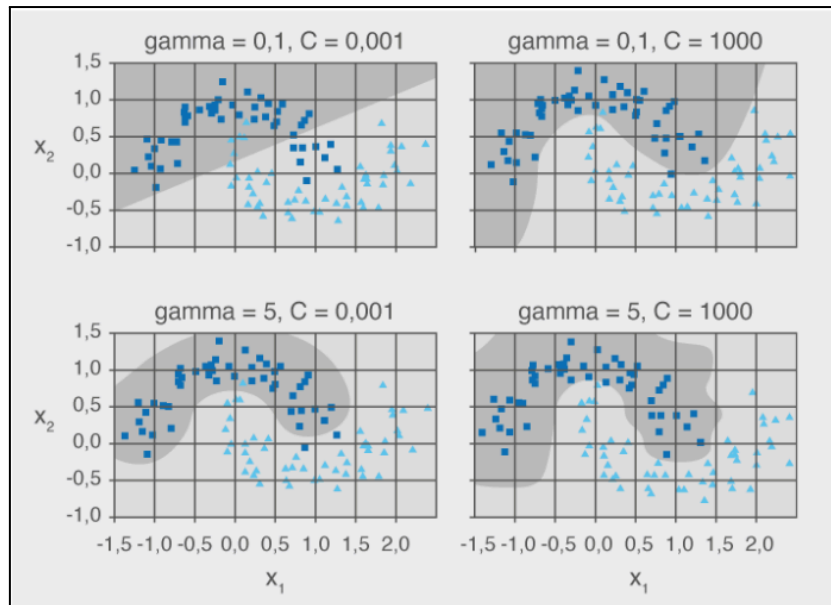
Los kernels más utilizados son:

- Lineal, que no aplica ninguna transformación y se usa cuando los datos ya son separables en el espacio original
- Polinómico, que transforma los datos con potencias para modelar relaciones suaves y no lineales
- RBF o gaussiano, que genera fronteras curvas ajustadas a la densidad de los datos, ideal para patrones complejos
- Sigmoide, inspirado en redes neuronales, aunque se utiliza con menos frecuencia en la práctica

Cuando se usan kernels no lineales en problemas de clasificación, aparecen dos parámetros adicionales que afectan directamente al comportamiento del modelo.

- Gamma controla hasta qué punto influye un punto concreto sobre la forma de la frontera. Si gamma es alto, el modelo se adapta mucho a cada punto, lo que puede llevar a sobreajuste. Si gamma es bajo, la influencia se reparte más y la frontera se suaviza, lo que puede provocar subajuste si el modelo no capta bien las diferencias entre clases.
- C regula cuánto penaliza el modelo los errores de clasificación. Un valor alto de C fuerza al modelo a clasificar bien todos los datos, lo que también puede causar sobreajuste. Un valor bajo permite errores a cambio de una frontera más simple y general, con menor riesgo de sobreajuste pero mayor riesgo de subajuste.

Por eso, para obtener buenos resultados con SVM, es importante probar distintas combinaciones de kernel, gamma y C, ajustándolos según el comportamiento del modelo en validación.



En esta imagen, con un kernel RBF, se muestran distintas combinaciones de los valores de gamma y C. Ambos parámetros influyen directamente en cómo se forma la frontera que separa las clases.

Cuando gamma es alto, cada punto tiene una influencia muy localizada y la frontera se adapta mucho a los detalles, haciéndose más irregular. Si gamma es bajo, esa influencia se extiende más y la frontera se vuelve más suave. Por eso, si el modelo está sobreajustado, puede ayudar reducir gamma, y si está subajustado, conviene aumentarlo.

El parámetro C controla cuánto se penalizan los errores. Un valor alto obliga al modelo a clasificar bien todos los datos, aunque genere una frontera compleja. Un valor bajo permite errores a cambio de una separación más general. Ajustar bien ambos es clave para encontrar un equilibrio entre precisión y generalización.

Árboles de decisión criterios de división y sobreajuste

Idea principal y ejemplos de uso

Los árboles de decisión son modelos de clasificación que buscan dividir los datos en grupos cada vez más homogéneos. A partir de las características de entrada, el modelo aprende una secuencia de reglas que permiten asignar una clase a cada observación.

Son especialmente útiles cuando se trabaja con datos categóricos, numéricos o mixtos, y cuando las distribuciones no siguen patrones lineales. Funcionan bien si las clases se pueden separar mediante condiciones simples, como rangos o igualdad de valores.

Se aplican en problemas como diagnóstico médico, análisis de riesgo, clasificación de clientes o detección de fraude, y en general cuando se necesita un modelo comprensible y rápido de interpretar.

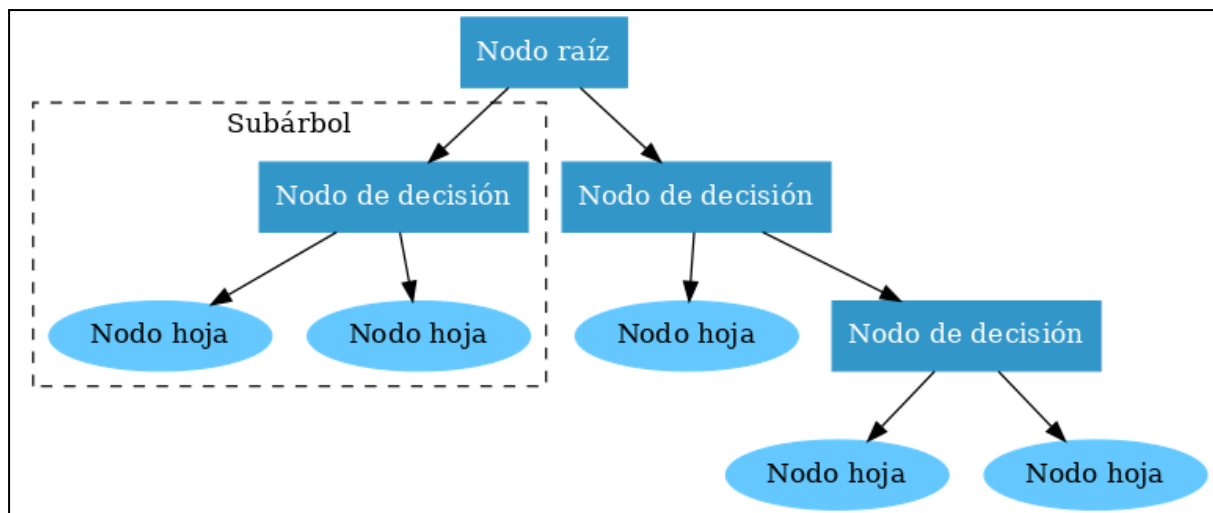
Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Cómo funciona

El modelo comienza con un nodo raíz, que contiene todos los datos. A partir de ahí, se crean divisiones sucesivas basadas en condiciones sobre las características que mejor separan las clases. Cada división genera nuevos nodos, y se repite el proceso de forma recursiva

Los nodos de decisión son aquellos que aún dividen los datos, mientras que los nodos hoja (o terminales) son aquellos en los que ya no se divide más y se asigna una clase final. Un subárbol es cualquier rama completa que parte de un nodo y contiene sus divisiones internas y nodos hoja.



Para decidir por dónde dividir, se utilizan criterios como el índice Gini o la entropía. Ambos miden la impureza de un grupo de datos.

El índice Gini es una medida de impureza o heterogeneidad utilizada para evaluar lo mezcladas que están las clases dentro de un nodo. Cuanto más homogéneo es un nodo, menor es su valor de Gini. Si todas las muestras pertenecen a una sola clase, el índice es 0, indicando pureza total. Durante la construcción del árbol, el algoritmo analiza todas las divisiones posibles y elige aquella que genera subgrupos más homogéneos, es decir, que más reduzca el valor de Gini. Esto permite que las decisiones del árbol sean más precisas, porque intenta que cada rama contenga datos lo más parecidos posible.

La entropía mide el grado de desorden o incertidumbre en un conjunto de datos. Si todas las muestras de un nodo son de la misma clase, la entropía es 0. Si hay una mezcla uniforme de clases, la entropía es máxima. Para decidir una división, el árbol calcula la ganancia de información, que es cuánto se reduce la entropía al dividir un nodo. Cuanto mayor es esta ganancia, mejor es la división. El objetivo es construir un árbol que reduzca progresivamente la incertidumbre y cree nodos cada vez más homogéneos.

El algoritmo utiliza un único criterio de impureza (como Gini o entropía) para todo el árbol. En cada paso, evalúa las posibles divisiones con ese criterio y elige la que más reduce la impureza de los grupos resultantes.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Puntos clave del algoritmo

- El árbol divide los datos en cada nodo aplicando condiciones simples sobre las características
- Utiliza un criterio de impureza (como Gini o entropía) para decidir la mejor división en cada paso
- Las divisiones se realizan de forma recursiva hasta cumplir una condición de parada o alcanzar nodos puros
- Cada hoja representa una clase final, y el camino desde la raíz hasta ella se interpreta como una secuencia de reglas
- El modelo puede volverse muy complejo si se deja crecer sin límites, lo que lleva a sobreajuste
- No necesita normalizar ni escalar las variables, ya que trabaja directamente con los valores
- Es sensible a pequeñas variaciones en los datos, lo que puede cambiar mucho la estructura del árbol
- Es un modelo fácilmente interpretable, ideal cuando se busca explicar las decisiones del sistema

Cuándo funciona bien y cuándo no

Los árboles de decisión funcionan especialmente bien cuando las clases se pueden separar mediante reglas simples basadas en condiciones sobre las características. Son útiles cuando hay variables categóricas o combinaciones lógicas claras, y cuando se busca un modelo interpretable y fácil de explicar. También se adaptan bien a conjuntos con ruido moderado y sin requerimientos de escalado.

En cambio, no funcionan tan bien cuando los límites entre clases son curvos o muy complejos, ya que las divisiones son siempre rectas y paralelas a los ejes. También tienden a sobreajustar si se les deja crecer sin control, sobre todo cuando hay mucho ruido o pocos datos. En problemas con muchas variables irrelevantes o correlacionadas, el árbol puede volverse inestable y difícil de generalizar.

Particularidades del algoritmo

Los árboles de decisión permiten ajustar su comportamiento a través de varios hiperparámetros que controlan su crecimiento y evitan el sobreajuste. Los más relevantes son:

- `max_depth`, que limita la profundidad máxima del árbol
- `min_samples_split`, que define el número mínimo de muestras para dividir un nodo
- `min_samples_leaf`, que exige un número mínimo de muestras en cada hoja
- `max_features`, que limita cuántas características se consideran en cada división
- `max_leaf_nodes`, que restringe el número total de hojas del árbol
- `min_impurity_decrease`, que impone una reducción mínima de impureza para aceptar una división
- `criterion`, que define la función para medir la calidad de una división (`gini` o `entropy`)

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Ajustar estos parámetros permite controlar el equilibrio entre la precisión y la capacidad de generalización del modelo. Un árbol con pocos límites tiende a memorizar los datos, mientras que uno demasiado restringido puede no captar patrones importantes.

Los árboles no necesitan escalar los datos ni hacer transformaciones previas. Funcionan bien con datos numéricos y categóricos, y su estructura basada en reglas los hace fáciles de entender. Sin embargo, son sensibles a pequeñas variaciones en los datos y pueden cambiar mucho con pocas muestras diferentes, por eso se suelen usar junto con métodos como los bosques aleatorios para ganar estabilidad.

Random Forest agregación de modelos e importancia de variables

Idea principal y ejemplos de uso

Random Forest es un algoritmo de conjunto que combina múltiples árboles de decisión para generar un modelo más robusto y preciso. Su fuerza radica en reducir el sobreajuste y aumentar la capacidad de generalización al trabajar con árboles independientes y diversos.

Se adapta bien a datos tabulares con muchas características, distribuciones no lineales, presencia de ruido o outliers, y relaciones complejas entre atributos. Es eficaz en tareas de clasificación y regresión como diagnóstico médico, predicción de precios, detección de fraude o clasificación de textos, incluso cuando hay muchas características irrelevantes o correlacionadas.

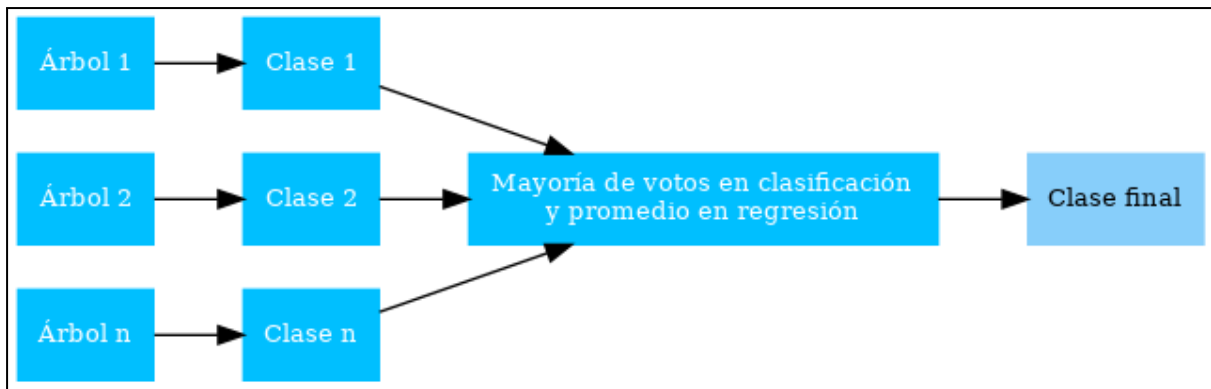
Cómo funciona

El algoritmo comienza creando varios subconjuntos aleatorios del conjunto de entrenamiento. Estos subconjuntos se generan con reemplazo, lo que significa que un mismo dato puede aparecer varias veces en diferentes muestras. A cada uno de estos subconjuntos se le construye un árbol de decisión propio, entrenado de forma independiente.

Durante la construcción de cada árbol, no se consideran todas las características para decidir las divisiones, sino que en cada nodo se elige aleatoriamente un subconjunto. Esto introduce variedad entre los árboles y evita que todos tomen decisiones similares.

Una vez entrenados todos los árboles, se utilizan en conjunto para hacer predicciones. En clasificación, cada árbol vota por una clase y la opción mayoritaria se toma como resultado final. En regresión, se calcula la media de las predicciones de todos los árboles.

Este proceso mejora la precisión del modelo, ya que cada árbol comete errores distintos, y al agregarlos se reducen los errores individuales. También ayuda a evitar el sobreajuste, ya que los árboles, al ser diferentes entre sí, no se adaptan en exceso al conjunto de entrenamiento.



Puntos clave del algoritmo

- Construye múltiples árboles independientes para reducir el error
- Introduce aleatoriedad en la selección de datos y características para aumentar la diversidad
- Usa votación mayoritaria en clasificación y promedio en regresión
- Permite medir la importancia relativa de las características
- Es robusto frente al ruido y valores atípicos
- Puede manejar grandes cantidades de datos y características sin necesidad de preprocesamiento complejo

Cuándo funciona bien y cuándo no

Los bosques aleatorios funcionan especialmente bien cuando hay muchas características y las relaciones entre ellas son complejas o no lineales. Son útiles en problemas con ruido, outliers o datos incompletos, ya que su capacidad para combinar múltiples árboles reduce el impacto de valores atípicos y errores puntuales.

Además, se adaptan bien a conjuntos con características correlacionadas o irrelevantes, porque el muestreo aleatorio en cada división favorece la diversidad y evita que un solo atributo domine el modelo.

En cambio, pueden no ser la mejor opción cuando se necesita un modelo fácilmente interpretable, ya que la combinación de muchos árboles dificulta la explicación directa. También requieren más recursos computacionales para entrenamiento y predicción, lo que puede ser un problema en sistemas con limitaciones de tiempo o memoria.

Particularidades del algoritmo

Random Forest destaca por varias características que lo hacen potente y eficiente en problemas complejos:

- **Diversidad entre árboles:** Para evitar que todos los árboles sean iguales y produzcan resultados redundantes, cada uno se construye usando solo un subconjunto aleatorio de características. Esto fomenta que cada árbol capture patrones diferentes en los datos, mejorando la precisión general y reduciendo la correlación entre árboles.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



- **Mitigación de la maldición de la dimensionalidad:** En conjuntos con muchas características, el hecho de no usar todas en cada división ayuda a que el modelo no se vea afectado negativamente por variables irrelevantes o ruido, facilitando la generalización.
- **Entrenamiento paralelizable:** Dado que cada árbol se construye de forma independiente, es posible entrenar múltiples árboles al mismo tiempo en diferentes núcleos o máquinas, acelerando notablemente el proceso y aprovechando el hardware disponible.
- **Resultados robustos y estables:** Al combinar las predicciones de muchos árboles a través de la votación mayoritaria (clasificación) o la media (regresión), Random Forest reduce la varianza del modelo y evita el sobreajuste típico de un único árbol muy profundo.
- **Estimación interna de error y confianza:** Utiliza el conjunto *out-of-bag* (OOB), que son las muestras no incluidas en el bootstrap de cada árbol, para evaluar su rendimiento sin necesidad de un conjunto de validación externo. Esto facilita ajustes rápidos y fiables.

Los hiperparámetros más relevantes són:

- **n_estimators:** determina la cantidad de árboles en el bosque. Más árboles suelen mejorar la estabilidad y la precisión, pero incrementan el coste computacional. A partir de cierto número (habitualmente sobre 100), la mejora es marginal.
- **max_features:** controla cuántas características se evalúan para decidir la división en cada nodo. Un valor bajo aumenta la diversidad entre árboles pero puede reducir la calidad individual de cada uno; un valor alto puede generar árboles más similares y menos robustos.
- **max_depth** y **min_samples_split:** regulan la complejidad de cada árbol, limitando la profundidad máxima y el mínimo número de muestras para dividir un nodo. Ajustar estos valores ayuda a controlar el sobreajuste y optimizar el rendimiento.
- **n_jobs:** especifica el número de núcleos de CPU usados para entrenar en paralelo. Usar todos los núcleos disponibles (-1) acelera el entrenamiento, especialmente con muchos árboles.

AdaBoost ponderación de errores y mejora iterativa

Idea principal y ejemplos de uso

AdaBoost es un algoritmo de ensamblado que combina múltiples clasificadores simples para construir un modelo final más robusto y preciso. En lugar de un único modelo complejo, utiliza una secuencia de modelos débiles que se corrigen entre sí.

Funciona bien con conjuntos de datos estructurados, es decir, aquellos organizados en filas y columnas con variables numéricas o categóricas. Es especialmente útil cuando las clases no están claramente separadas, como en distribuciones donde los datos de distintas clases se mezclan o los límites son difíciles de definir. También puede manejar cierto nivel de ruido si hay suficientes datos.

Se usa habitualmente en tareas como detección de spam, predicción de fraude, reconocimiento facial o diagnóstico médico, donde mejorar la precisión sin construir modelos excesivamente complejos es una prioridad.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)

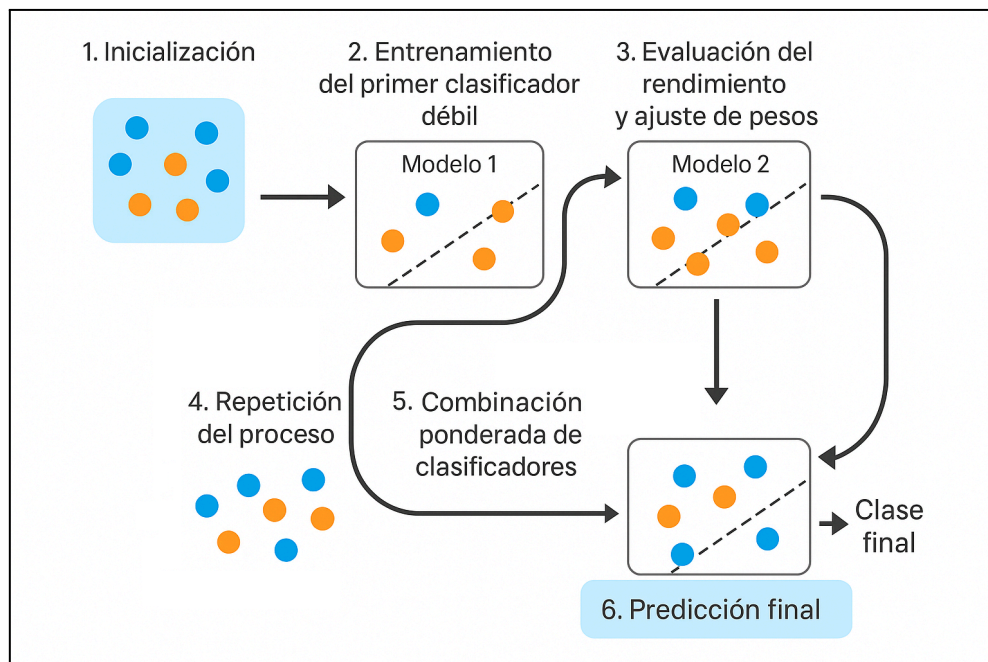


Cómo funciona

AdaBoost mejora iterativamente la precisión del modelo. En cada paso, identifica los datos mal clasificados y les da más peso para que el siguiente clasificador se centre en ellos. Así, los errores se corrigen poco a poco.

El proceso completo funciona así:

1. **Inicialización:** Todas las muestras del conjunto de entrenamiento comienzan con el mismo peso. Es el punto de partida antes de entrenar el primer modelo.
2. **Entrenamiento del primer clasificador débil:** Se entrena un modelo simple (por ejemplo, un árbol de decisión poco profundo) sobre los datos ponderados. Este modelo intenta clasificar correctamente, pero comete errores.
3. **Evaluación del rendimiento y ajuste de pesos:** Se identifica qué observaciones han sido clasificadas incorrectamente. Sus pesos se incrementan para que tengan más influencia en el siguiente modelo.
4. **Repetición del proceso:** Se entrena un nuevo clasificador débil teniendo en cuenta los nuevos pesos. Este ciclo se repite durante varias iteraciones, cada vez corrigiendo los errores del anterior.
5. **Combinación ponderada de clasificadores:** Cada modelo entrenado recibe un peso que depende de su precisión. Los modelos más fiables tienen más peso en la decisión final.
6. **Predicción final:** Se combinan todas las predicciones mediante una votación ponderada. La clase final que se asigna a una nueva observación es la que recibe más apoyo entre los modelos, según su peso.



Puntos clave del algoritmo

- Combina varios clasificadores débiles entrenados de forma secuencial

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



- Cada nuevo modelo se centra en corregir los errores de los anteriores
- Asigna más peso a las muestras mal clasificadas en cada iteración
- Los clasificadores más precisos tienen mayor influencia en la predicción final
- La combinación se realiza mediante una suma ponderada de los modelos individuales
- Utiliza clasificadores base muy simples, como árboles poco profundos (stumps)
- Tiene pocos hiperparámetros y no requiere una optimización compleja
- Puede evitar el sobreajuste limitando el número de iteraciones (`n_estimators`)
- El aprendizaje es secuencial, por lo que no se puede paralelizar fácilmente
- Es sensible al ruido o a los outliers, ya que estos reciben más peso en el proceso
- Funciona bien cuando las clases no se separan claramente y la frontera es compleja o no lineal

Cuándo funciona bien y cuándo no

AdaBoost funciona especialmente bien cuando se utilizan clasificadores simples sobre datos con patrones complejos o fronteras no lineales, incluso si la separación entre clases no es perfecta. Es eficaz con datos tabulares, ya sean numéricos o categóricos, y en tareas donde se busca mejorar el rendimiento sin recurrir a modelos pesados. Resulta útil cuando se dispone de un conjunto de datos relativamente limpio y equilibrado, y se quiere mejorar la precisión sin un alto riesgo de sobreajuste.

En cambio, su rendimiento disminuye en presencia de ruido, outliers o errores en los datos, ya que tiende a dar más peso a las muestras mal clasificadas, lo que puede llevar al sobreajuste. También puede ser menos eficaz en situaciones con clases muy desbalanceadas o cuando los clasificadores base no son capaces de extraer patrones relevantes, aunque se les asigne más peso. Además, al ser un método secuencial, no escala bien con conjuntos de datos muy grandes o de alta dimensionalidad si no se optimiza adecuadamente.

Particularidades del algoritmo

AdaBoost cuenta con pocos hiperparámetros, pero su configuración influye directamente en la calidad y generalización del modelo. No requiere validación cruzada interna para ajustar los pesos, aunque es recomendable evaluar el rendimiento con un conjunto de validación externo.

Los hiperparámetros más relevantes son:

- `n_estimators`: indica cuántos clasificadores débiles se entrenan de forma secuencial. Aumentar su valor mejora la capacidad del modelo, pero también eleva el tiempo de entrenamiento y puede favorecer el sobreajuste si los datos contienen ruido.
- `learning_rate`: regula la influencia de cada clasificador débil en la predicción final. Valores bajos producen un aprendizaje más gradual y estable; valores altos aceleran el proceso, pero pueden afectar la capacidad de generalización.
- `base_estimator`: define el modelo utilizado como clasificador débil. Por defecto es un árbol de decisión con profundidad máxima de 1 (stump), aunque puede sustituirse por otros modelos como regresión logística o SVM lineal.
- `algorithm`: especifica el método de ajuste de pesos entre iteraciones:
 - **SAMME**: usa directamente las predicciones de clase, útil cuando los clasificadores base no generan probabilidades.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- **SAMME.R**: emplea probabilidades para ajustar los pesos de forma más fina. Suele dar mejores resultados si el modelo base puede estimarlas.

AdaBoost utiliza una función de pérdida exponencial que penaliza con más fuerza los errores cometidos con alta confianza. Esta estrategia permite centrarse en las muestras más difíciles, pero también incrementa la sensibilidad a datos ruidosos o valores atípicos.

Comparación general de algoritmos

Algoritmo	Objetivo general	Ventajas	Limitaciones	Hiperparámetros más importantes	Puntos clave a tener en cuenta
Naive Bayes	Estimar la clase más probable asumiendo independencia condicional entre características	Rápido, robusto al ruido, eficaz en alta dimensionalidad. Ideal con datos categóricos o de texto. Útil en conteos o distribuciones discretas. No necesita muchos datos para funcionar bien.	Baja precisión si las características no son independientes. No capta relaciones entre atributos	Tipo de distribución (Gaussiana, Multinomial, Bernoulli)	No requiere escalado para variables numéricas. Acepta categóricas sin codificar. Muy eficiente incluso con pocos datos.
KNN	Clasificar en función de los ejemplos más cercanos en el espacio de características	Sencillo, sin entrenamiento previo. Captura bien formas irregulares y fronteras no lineales	Sensible a outliers y ruido. Costoso en predicción. Escala mal con muchos datos o dimensiones	k (número de vecinos), métrica de distancia, ponderación por distancia	Requiere escalado para numéricas. No acepta categóricas sin codificar. Afectado por la métrica y la normalización.
Regresión logística	Modelar la probabilidad de pertenencia a una clase en base a una combinación	Precisa con relaciones lineales, fácil de interpretar. Eficiente en datos tabulares	Baja capacidad para capturar relaciones no lineales o complejas.	C (regularización), tipo de penalización (L1, L2), solver	Requiere escalado para numéricas. No acepta categóricas sin codificar. Asume

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



	lineal de características	con distribución sencilla	Afectada por multicolinealidad		relación lineal con log-odds.
SVM	Encontrar el hiperplano óptimo que maximiza el margen entre clases	Eficaz en alta dimensionalidad. Buen rendimiento con clases separables y márgenes claros	Difícil de ajustar. Sensible a ruido y solapamiento. Lenta en conjuntos grandes	C (regularización), kernel, gamma, margen suave	Requiere escalado para numéricas. No acepta categóricas. Cuidado con el kernel y la selección de parámetros.
Árboles de decisión	Crear reglas de decisión simples a partir de divisiones sucesivas en las características	Maneja bien datos mixtos y valores faltantes. Interpretable. Buen rendimiento con combinaciones lógicas	Sobreajuste frecuente con ruido o árboles profundos. Fronteras siempre rectas	max_depth, min_samples_split, min_samples_leaf, criterion	No requiere escalado. Acepta categóricas sin necesidad de codificación. Sensible a sobreajuste si no se controla profundidad.
Random Forest	Mejorar la precisión mediante la agregación de muchos árboles diversos entrenados con aleatoriedad	Robusto a ruido y outliers. Maneja correlaciones y alta dimensionalidad. Se adapta bien a datos tabulares	Difícil de interpretar. Más costoso en entrenamiento. A partir de cierto número, añadir árboles no mejora	n_estimators, max_features, max_depth, n_jobs	No requiere escalado. Acepta categóricas. Evalúa importancia de características. Entrena en paralelo.
AdaBoost	Potenciar el rendimiento de clasificadores débiles corrigiendo errores de forma iterativa	Preciso con datos complejos y separación no perfecta. Bueno con datos numéricos o categóricos. Pocos hiperparámetros	Muy sensible a outliers y ruido. Entrenamiento secuencial no paralelizable. Puede sobreajustar con datos erróneos	n_estimators, learning_rate, base_estimator, algorithm	Requiere escalado si el estimador base lo necesita. Acepta categóricas si el clasificador base lo permite.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



					Sensible al ruido.
--	--	--	--	--	--------------------

Notas adicionales

- Aunque algunos algoritmos como Naive Bayes, Árboles de Decisión o Random Forest no requieren escalado, aplicarlo puede ser conveniente si se combinan con otros modelos o para mantener coherencia en el preprocesamiento.
- Aceptar datos categóricos no implica que no se beneficien de una codificación adecuada, sobre todo cuando hay muchas categorías poco frecuentes o si se desea controlar su impacto en el modelo.
- En métodos como KNN, SVM o Regresión Logística, el escalado es esencial para evitar que variables con mayor rango dominen la predicción. Se recomienda usar MinMaxScaler o StandardScaler.
- En AdaBoost, si se usa un estimador base que requiere normalización (como SVM o regresión logística), será necesario escalar también las características numéricas.
- Naive Bayes no es adecuado si hay fuerte correlación entre variables. KNN y SVM son sensibles al ruido y a valores extremos. Árboles y Random Forest manejan bien atributos irrelevantes.
- En conjuntos con alta multicolinealidad, métodos basados en árboles (como Random Forest o AdaBoost) suelen ser más robustos que modelos lineales como SVM o regresión.

Importante: Cada problema es distinto y siempre será necesario ajustar, validar y optimizar los modelos según el contexto, los datos disponibles y el objetivo del análisis.

Lección 3. Modelos supervisados para regresión: algoritmos, métricas y usos

Papel de la regresión, ejemplos de aplicación práctica

La regresión es una técnica fundamental en aprendizaje automático supervisado orientada a predecir valores numéricos continuos a partir de variables conocidas. A diferencia de la clasificación, donde se asigna una etiqueta, en regresión el objetivo es estimar una cantidad, como una puntuación, una duración, un coste o una probabilidad.

Este tipo de modelos es muy útil en situaciones donde se necesita prever comportamientos, anticipar resultados o tomar decisiones basadas en magnitudes. La regresión se aplica en campos muy diversos, desde la predicción de resultados académicos, el análisis del consumo energético, la estimación de tiempos de espera o entrega, hasta la modelización de riesgos financieros. También puede usarse en contextos donde interesa calcular precios o rendimientos, como en el caso del mercado inmobiliario, pero este es solo uno de los muchos posibles escenarios.

Para evaluar la calidad de las predicciones no basta con saber si se acercan o no al valor real. Dependiendo del problema, pueden interesar distintos tipos de error. Por eso se utilizan métricas específicas como el error cuadrático medio, el error absoluto medio o el coeficiente de determinación. Estas métricas permiten entender mejor cómo se comporta el modelo en diferentes rangos de valores o si tiende a sobrestimar o subestimar los resultados.

Métricas para evaluar regresión

MAE, MSE, RMSE, comparación y sensibilidad a valores extremos

Cuando se trabaja con modelos de regresión, es fundamental evaluar qué tan cerca están las predicciones del modelo respecto a los valores reales. Para ello se utilizan métricas específicas de error. Las tres más comunes son:

MAE (Error Absoluto Medio)

Calcula la media de las diferencias absolutas entre los valores reales y los predichos. Es decir, mide cuánto se equivoca el modelo en promedio, sin importar si lo hace por arriba o por abajo.

Por ejemplo, si el modelo predice que un coche cuesta 20.000 €, pero en realidad cuesta 22.000 €, la diferencia absoluta es de 2.000 €. Si este tipo de error se repite varias veces, el MAE reflejará ese promedio de forma directa, sin penalizar más los errores grandes.

Es una métrica útil cuando todos los errores tienen la misma importancia, independientemente de si son pequeños o grandes. Se suele usar en contextos donde queremos minimizar el error medio general y donde los valores atípicos no deben influir demasiado, como en estimaciones de consumo, temperaturas o tiempos de entrega.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



MSE (Error Cuadrático Medio)

Es similar al MAE, pero en lugar de tomar el valor absoluto de cada error, eleva las diferencias al cuadrado. Esto provoca que los errores grandes tengan mucho más peso en el resultado final, ya que su impacto se multiplica.

Por ejemplo, si un modelo se equivoca en 500 €, su error cuadrático será 250.000. Si el error es de 2.000 €, el resultado será 4.000.000. Es decir, aunque el segundo error es cuatro veces mayor, su penalización es dieciséis veces más alta. Esta característica hace que el MSE destaque mucho más los errores grandes.

Por eso, es una métrica útil cuando queremos que el modelo evite desviaciones grandes a toda costa. A cambio, puede ser muy sensible a valores atípicos, ya que los penaliza con mucha fuerza.

Se recomienda utilizar el MSE cuando los errores grandes tienen consecuencias más importantes que los pequeños, por ejemplo, en predicción de precios de viviendas, costes médicos o estimaciones económicas donde un error elevado puede tener un gran impacto práctico.

RMSE (Raíz del Error Cuadrático Medio)

Es muy parecido al MSE, pero después de calcular el error cuadrático medio, se aplica una raíz cuadrada. Esto hace que el resultado vuelva a estar en las mismas unidades que la variable a predecir, lo que facilita su interpretación.

Por ejemplo, si un modelo predice que un piso cuesta 180.000 €, pero en realidad cuesta 200.000 €, el error es de 20.000 €. Si este tipo de error se repite, el RMSE dará una media en euros, como el precio original.

El RMSE también penaliza más los errores grandes, igual que el MSE, pero al aplicar la raíz cuadrada se suaviza el efecto y resulta más legible.

Es útil cuando se quiere penalizar los errores grandes, pero se necesita que el resultado esté en la misma escala que los datos. Muy usado cuando importa mucho la precisión y se quiere interpretar el margen medio de error fácilmente.

Cada una de estas métricas responde de forma diferente ante errores grandes.

El MAE da el mismo peso a todos los errores, sean pequeños o grandes. Es una opción equilibrada si se quiere medir la precisión media sin dejar que unos pocos errores extremos influyan demasiado. Por ejemplo, si estamos ajustando un modelo para predecir el precio medio de productos de supermercado, donde pequeñas desviaciones son tolerables y los valores extremos son raros, el MAE puede ser más representativo de la calidad del modelo.

El MSE y el RMSE, en cambio, penalizan más los errores grandes porque elevan al cuadrado las diferencias. Esto hace que una predicción muy alejada del valor real tenga un impacto mucho mayor en el resultado final. Por eso son métricas más sensibles a valores atípicos. Son útiles cuando esos

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



errores grandes son especialmente importantes, como en la predicción del coste de una cirugía o de la inversión en bolsa, donde una mala predicción puntual puede tener un gran impacto económico.

Elegir una u otra depende del contexto. Si los errores grandes son críticos y se quieren evitar a toda costa, conviene usar MSE o RMSE. Si, en cambio, se prefiere una medida más robusta y estable frente a valores extremos, el MAE es la mejor opción.

R² y coeficiente de determinación, interpretación y límites

El coeficiente de determinación, conocido como R², indica qué proporción de la variabilidad de los valores reales puede explicarse por el modelo. Su valor suele situarse entre 0 y 1, aunque en algunos casos puede ser negativo. Cuanto más alto es, mejor encaja el modelo con los datos. Por ejemplo, un R² de 0.85 significa que el 85% de las diferencias entre los precios reales de unas casas se pueden explicar a partir de las variables utilizadas, como la superficie o el número de habitaciones.

Un R² de 1 indicaría que el modelo predice perfectamente todos los valores, mientras que un R² de 0 implica que el modelo no aporta ninguna mejora respecto a simplemente predecir la media. Si el valor es negativo, es señal de que el modelo está funcionando peor que una predicción constante basada en la media de los datos.

Esta métrica resulta útil cuando se quiere saber cuánto explica un modelo sobre el comportamiento general de los datos, pero no indica si las predicciones son realmente cercanas a los valores reales. Un modelo puede tener un R² alto y aun así cometer errores importantes en valores concretos. Por eso se recomienda usarla siempre junto con métricas como el MAE o el RMSE, que sí reflejan los errores en unidades reales.

R² también puede ser engañoso cuando el modelo tiene muchas variables, ya que su valor puede aumentar aunque las nuevas variables no aporten valor real. Para estos casos existe el R² ajustado, que tiene en cuenta la complejidad del modelo. Además, en problemas con patrones no lineales, puede no ser fiable si no se adapta bien el tipo de modelo.

En definitiva, el R² ayuda a entender qué porcentaje de la variabilidad de los datos estamos explicando, pero no debe utilizarse como único criterio para valorar un modelo de regresión. Sirve para comparar modelos aplicados al mismo problema, pero debe interpretarse con cuidado y complementarse siempre con otras métricas.

A veces, un modelo con un R² ligeramente más bajo puede ser preferible si el RMSE disminuye de forma considerable. Por ejemplo, si un modelo tiene un R² de 0.89 pero un RMSE de 5.000 euros, y otro tiene un R² de 0.85 pero un RMSE de 3.000 euros, es probable que el segundo sea más útil en la práctica, porque comete errores menores en promedio, aunque explique un poco menos de la variabilidad total. Esto es especialmente importante cuando se trabaja con datos sensibles, como precios o cantidades económicas, donde los errores grandes pueden tener más impacto que una ligera pérdida de capacidad explicativa.

Funciones de pérdida, error absoluto, error cuadrático, entropía cruzada adaptada

En los modelos de regresión, las funciones de pérdida son fórmulas que indican cuánto se está equivocando el modelo durante el entrenamiento. No se utilizan solo para evaluar el rendimiento final, sino también para guiar el proceso de aprendizaje. Cada vez que el modelo hace una predicción incorrecta, la función de pérdida calcula la magnitud del error y genera una señal que sirve para ajustar los parámetros internos del modelo (como los coeficientes en una regresión o los nodos en un árbol).

Estos ajustes no afectan a los hiperparámetros, que son valores definidos antes del entrenamiento (como la tasa de aprendizaje o la profundidad máxima del árbol), sino a los parámetros que el modelo va aprendiendo a medida que analiza los datos. Por tanto, una función de pérdida adecuada permite que el modelo se adapte progresivamente para reducir sus errores, mientras que una mal escogida puede llevarlo a sobreajustar, ignorar patrones importantes o tener mal rendimiento en datos nuevos.

Error absoluto (EAM, Error Absoluto Medio)

Esta función mide la diferencia entre el valor real y el valor predicho, sin importar si el error es positivo o negativo. Se le conoce también como *L1 loss*. Es útil cuando se desea una penalización uniforme para todos los errores, ya que trata igual los pequeños y los grandes. Por ejemplo, si un modelo predice 155.000 € y el valor real es 150.000 €, el error absoluto será de 5.000 €.

Error cuadrático (EQM, Error Cuadrático Medio)

También llamado *L2 loss*, eleva al cuadrado la diferencia entre valor real y valor predicho. Esto hace que los errores grandes tengan un peso mucho mayor. Se usa cuando se quiere evitar errores extremos. Siguiendo el ejemplo anterior, un error de 5.000 € pasaría a ser 25.000.000 al elevarse al cuadrado, por lo que el modelo intentará reducir especialmente los errores más grandes.

Ejemplo – Comparación entre EAM y EQM

Imagina un modelo que predice el valor de coches usados. Si un coche vale realmente 10.000 € y el modelo predice 9.000 €, el error es de 1.000 €. Ahora imagina que para otro coche el valor real es 15.000 €, pero el modelo predice 10.000 €, cometiendo un error de 5.000 €.

Si usamos EAM (Error Absoluto Medio), ambos errores se suman sin distinción especial y se promedian. El resultado reflejará un error medio de 3.000 €, tratando los errores por igual. Este enfoque es útil cuando todos los errores son importantes por igual y no queremos que los más grandes influyan demasiado.

En cambio, si usamos EQM (Error Cuadrático Medio), el error de 1.000 € se convierte en 1.000.000, pero el de 5.000 € pasa a ser 25.000.000. Esto hace que el error más grande tenga mucha más influencia sobre la media, elevando el EQM a un valor mucho mayor. Por eso, el EQM se usa cuando se quiere penalizar especialmente los errores grandes y evitar que el modelo se equivoque mucho en casos puntuales.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Entropía cruzada adaptada

Aunque esta función es más común en clasificación, también puede adaptarse a regresión si los valores continuos se agrupan en intervalos o clases. La entropía cruzada (*cross-entropy*) mide la diferencia entre la distribución de probabilidad real y la predicha. Por ejemplo, si se quiere predecir la temperatura en rangos (baja, media, alta), se puede usar entropía cruzada para comparar cuán bien se ajusta la probabilidad estimada con la clase real. Es útil en problemas de regresión que se reformulan como clasificación con rangos definidos.

Ejemplo – Uso de entropía cruzada adaptada

Supón que quieres predecir la temperatura ambiental, pero en lugar de un valor exacto, te interesa saber si será baja, media o alta, ya que es más útil para activar sistemas de ventilación o calefacción.

En este caso, puedes dividir los valores continuos de temperatura en tres rangos y tratar el problema como una clasificación. Si el modelo predice que hay un 60 % de probabilidad de que la temperatura sea “alta”, un 30 % “media” y un 10 % “baja”, y el valor real cae en el rango “alta”, el error será pequeño. Pero si el modelo hubiera asignado un 10 % a “alta” y un 80 % a “baja”, la entropía cruzada sería mucho mayor, porque la predicción está muy lejos de la realidad.

Este enfoque permite trabajar con distribuciones de probabilidad, y es especialmente útil cuando lo importante es anticipar correctamente el nivel o categoría en la que cae el valor, más que la cifra exacta.

Comprender la diferencia entre precisión y exactitud en regresión

Al evaluar modelos de regresión, no solo interesa saber cuánto se equivocan, sino también cómo se comportan sus predicciones entre sí. Dos conceptos que ayudan a entender esto son la **precisión** y la **exactitud**, que aunque suelen asociarse a tareas de clasificación, también tienen sentido en regresión.

En este caso, hablamos de **exactitud** cuando las predicciones se aproximan mucho a los valores reales. En cambio, un modelo es **preciso** si sus predicciones son coherentes entre sí, aunque estén lejos del valor correcto. Es decir, la precisión se relaciona con la dispersión interna del modelo y la exactitud con su capacidad para acertar.

Por ejemplo, si un modelo predice siempre 155.000€ para viviendas que en realidad cuestan 120.000€, 190.000€ y 155.000€, podemos decir que es muy preciso, porque repite siempre el mismo valor, pero poco exacto, porque solo acierta en un caso y se equivoca bastante en los otros dos.

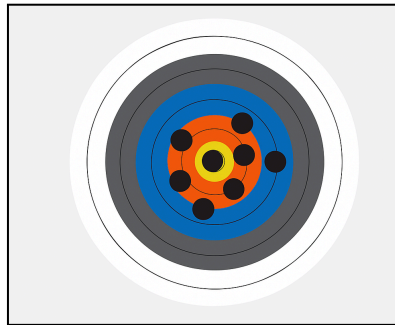
Este equilibrio entre coherencia y acierto se representa de forma muy clara en la imagen siguiente, que compara distintos escenarios de precisión y exactitud con la idea de un sistema de puntería.

Alta exactitud y baja precisión

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)

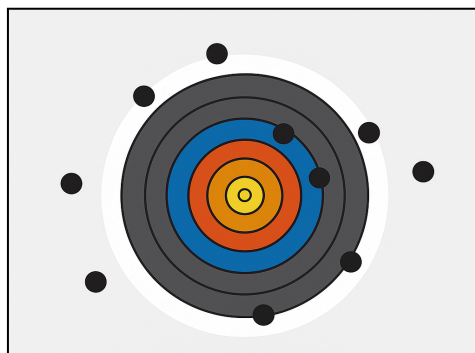


En este gráfico se observa que los puntos (las predicciones del modelo) están repartidos alrededor del centro del objetivo, pero no están agrupados entre sí. Esto significa que el modelo, en promedio, acierta bastante en su estimación (alta exactitud), pero presenta una gran variabilidad entre una predicción y otra (baja precisión). Es decir, aunque las predicciones se acercan al valor real, no son consistentes entre ellas. Este comportamiento puede ser aceptable si interesa un buen promedio, pero puede resultar problemático si se necesita estabilidad en cada predicción individual.



Baja exactitud y baja precisión

En esta imagen, los puntos están dispersos y alejados del centro de la diana. Esto indica que las predicciones del modelo ni son consistentes entre sí (baja precisión), ni se acercan al valor real (baja exactitud). Es el peor de los casos: el modelo no solo se equivoca, sino que lo hace de forma poco uniforme. Una situación así sugiere que el modelo no está captando bien la relación entre las variables y necesita una revisión profunda o un enfoque diferente.

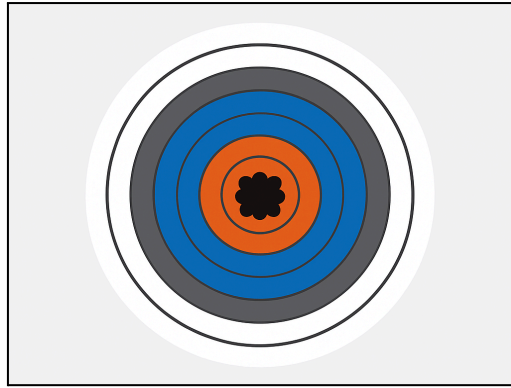


Alta exactitud y alta precisión

En esta representación vemos un caso de alta exactitud y alta precisión. Todos los puntos están agrupados muy cerca entre sí y, además, se sitúan justo en el centro de la diana. Esto indica que el modelo genera predicciones muy consistentes (alta precisión) y que esas predicciones están muy próximas al valor real (alta exactitud). Es el escenario ideal en problemas de regresión, ya que implica que el modelo no solo es fiable en sus resultados, sino también acertado.

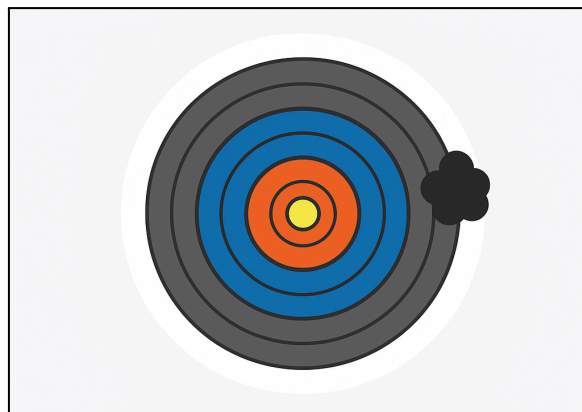
Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)





Baja exactitud y alta precisión

Este ejemplo representa un caso de alta precisión pero baja exactitud. Las predicciones del modelo están muy agrupadas entre sí (alta precisión), lo que indica consistencia, pero todas están alejadas del centro de la diana (baja exactitud), es decir, del valor real. Esto puede suceder cuando el modelo aprende un patrón erróneo o tiene un sesgo sistemático. Aunque siempre se equivoca de forma similar, no está acertando en el valor correcto.



Detección del sobreajuste en clasificación y análisis visual

El sobreajuste en regresión comparte el mismo principio que en clasificación: el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad para generalizar con datos nuevos. Al igual que en clasificación, se pueden utilizar técnicas como la validación cruzada, la comparación entre entrenamiento y test, o las curvas de aprendizaje para detectar este comportamiento. Sin embargo, la regresión también tiene herramientas propias que permiten analizar mejor cómo se comporta el modelo frente a valores numéricos continuos.

Una señal habitual de sobreajuste en regresión es obtener errores muy bajos sobre el conjunto de entrenamiento y errores significativamente mayores en validación o test. Pero para entender bien este fenómeno, no basta con observar las métricas: es necesario visualizar cómo predice el modelo y cómo se distribuyen los errores.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Evaluación directa de las predicciones

Una forma sencilla de detectar si un modelo está sobreajustado consiste en revisar las primeras predicciones una a una y compararlas con los valores reales. No basta con conocer el error medio. Al revisar caso por caso, se puede ver si el modelo tiene desviaciones sistemáticas, si falla en valores bajos o altos, o si comete errores grandes que podrían pasar desapercibidos en una métrica global. Observar también el porcentaje de error relativo ayuda a identificar si los errores son proporcionales o si el modelo se desajusta más en ciertos rangos.

Este análisis se puede complementar con un criterio práctico basado en márgenes de error relativos. Aunque no existe una norma teórica universal que establezca umbrales exactos, en algunos contextos se utilizan tramos orientativos para facilitar la interpretación. Por ejemplo, se puede considerar una predicción como buena si el error relativo es inferior al 10%, aceptable entre el 10% y el 25%, y mejorable si supera ese valor. Estos porcentajes no provienen de una teoría formal, sino que sirven como guía aproximada, y deben adaptarse según la naturaleza del problema y el impacto que tenga cada error en el contexto real.

A continuación se muestra un ejemplo con las primeras predicciones del conjunto de test. En cada caso se compara el valor real, la predicción, la diferencia y el porcentaje de error:

```
1: Real Time: 90.81 - Predicción Time: 78.84 - Diferencia: 11.97 - Error: 13.18%
2: Real Time: 15.67 - Predicción Time: 25.27 - Diferencia: -9.60 - Error: 61.27%
3: Real Time: 56.23 - Predicción Time: 72.97 - Diferencia: -16.74 - Error: 29.78%
4: Real Time: -3.57 - Predicción Time: -3.03 - Diferencia: -0.54 - Error: 15.11%
5: Real Time: 65.74 - Predicción Time: 64.62 - Diferencia: 1.12 - Error: 1.71%
6: Real Time: 47.95 - Predicción Time: 38.15 - Diferencia: 9.80 - Error: 20.44%
7: Real Time: 20.70 - Predicción Time: 28.64 - Diferencia: -7.94 - Error: 38.36%
8: Real Time: 24.15 - Predicción Time: 32.02 - Diferencia: -7.87 - Error: 32.60%
9: Real Time: 44.75 - Predicción Time: 58.18 - Diferencia: -13.43 - Error: 30.01%
10: Real Time: 53.09 - Predicción Time: 46.76 - Diferencia: 6.33 - Error: 11.92%
11: Real Time: 69.13 - Predicción Time: 71.71 - Diferencia: -2.58 - Error: 3.74%
12: Real Time: 1.38 - Predicción Time: 14.12 - Diferencia: -12.74 - Error: 923.10%
```

Este tipo de revisión permite identificar con precisión en qué casos concretos el modelo falla más, si los errores son razonables o inaceptables, y si hay algún patrón que se repite. Es especialmente útil en contextos donde se requiere precisión en la toma de decisiones, como predicción de tiempos, costes o variables médicas.

A continuación se muestra gráficamente esa misma información, comparando los valores reales y predichos de las primeras 50 muestras. Se puede observar si el modelo mantiene una diferencia constante, si tiende a sobreestimar o infraestimar, o si hay puntos con errores especialmente elevados:

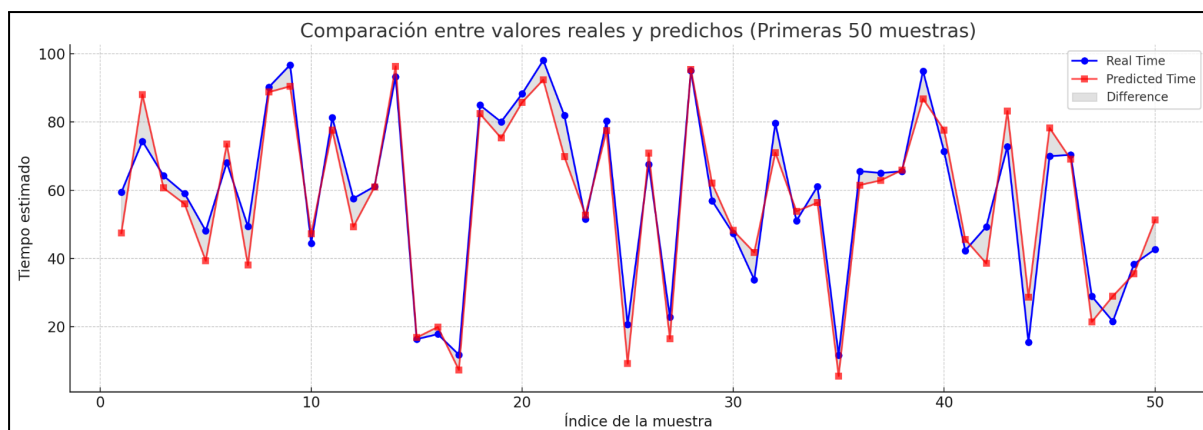


Gráfico de predicciones frente a valores reales

Este gráfico muestra la relación entre los valores predichos por el modelo y los valores reales esperados. En un modelo ideal, todos los puntos se situarían sobre una línea diagonal, indicando que la predicción es igual al valor real. Cuanto más dispersos estén los puntos respecto a esta línea, mayor será el error. Si se observa un patrón de desviación —por ejemplo, si el modelo sistemáticamente subestima los valores altos— puede indicar un problema de sobreajuste o una falta de capacidad para captar relaciones complejas.

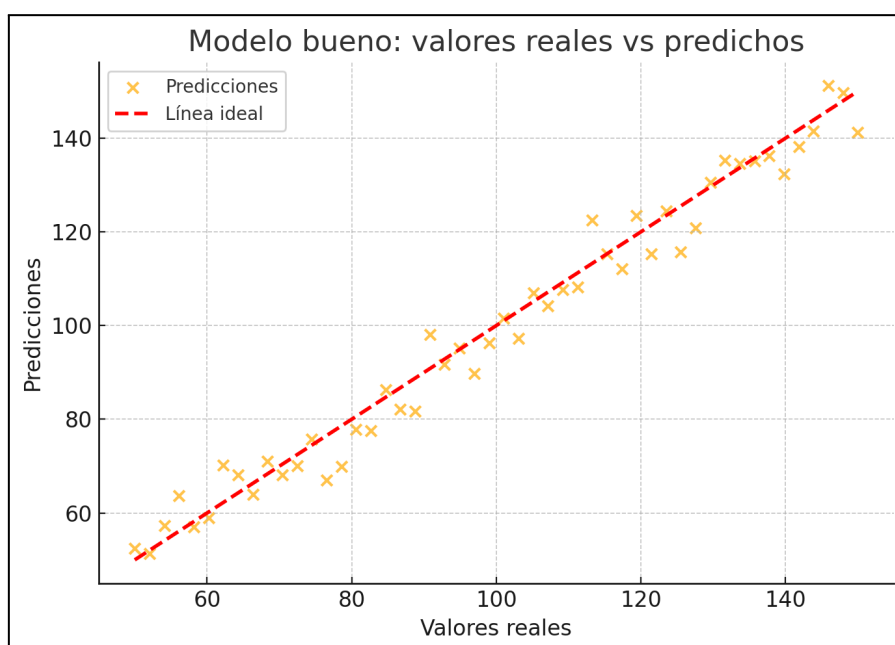


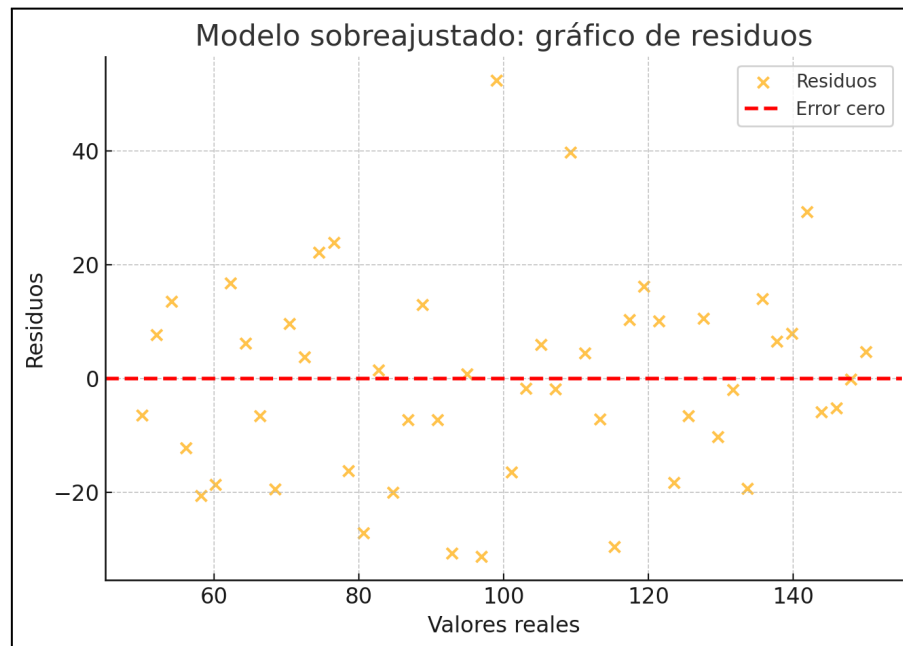
Gráfico de residuos

El gráfico de residuos representa la diferencia entre el valor real y el valor predicho. Es una herramienta clave en regresión para detectar errores no visibles en las métricas numéricas. En un modelo que generaliza bien, los residuos deberían distribuirse aleatoriamente alrededor de cero. Si aparecen agrupaciones, formas curvas o una dispersión no homogénea, podría significar que el

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)

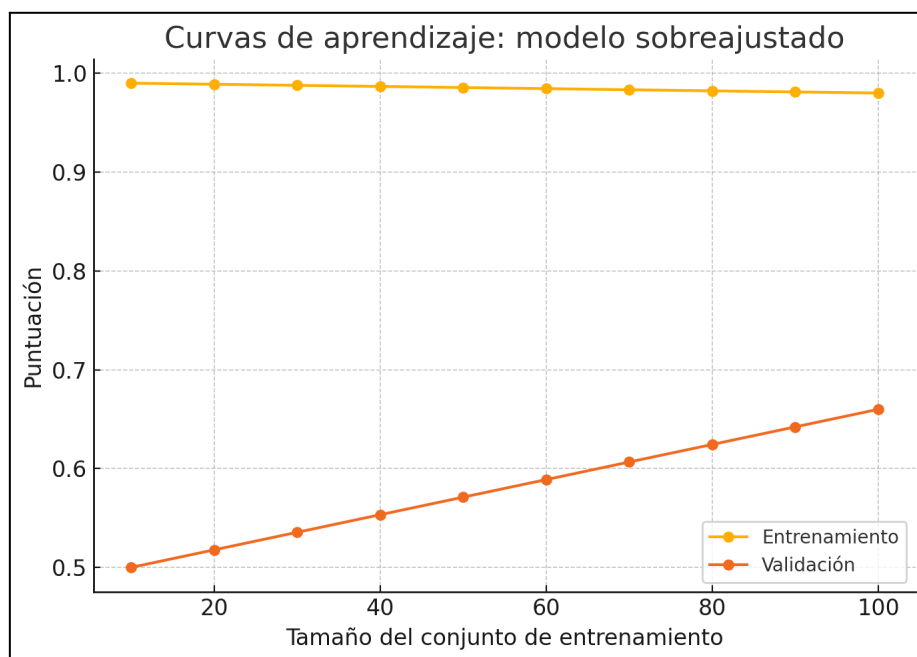
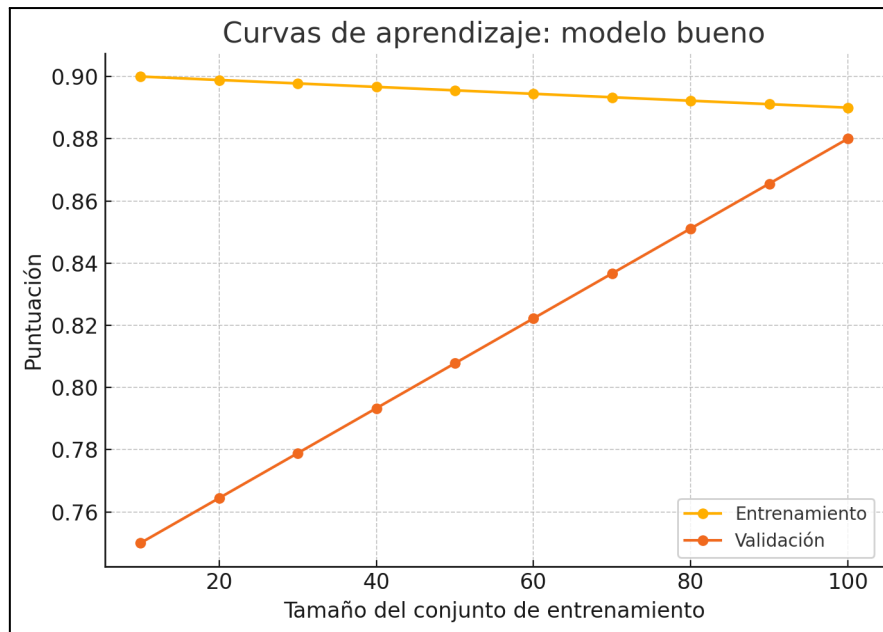


modelo no está captando bien la estructura de los datos o que está sobreajustado. Un error sistemático en los residuos suele ser una señal clara de que el modelo no generaliza correctamente.



Curvas de aprendizaje en regresión

Las curvas de aprendizaje permiten observar cómo evoluciona el rendimiento del modelo a medida que aumenta la cantidad de datos disponibles. En un modelo equilibrado, la puntuación en entrenamiento disminuye ligeramente mientras que la de validación mejora, hasta que ambas se estabilizan cerca. Si la puntuación en entrenamiento es muy alta pero la de validación queda muy por debajo, es probable que el modelo esté sobreajustando. En cambio, si ambas puntuaciones son bajas, el modelo puede estar infraajustado, lo que también se detecta visualmente. Observar la forma de las curvas y su distancia ayuda a tomar decisiones sobre el tipo de modelo y el volumen de datos necesario.



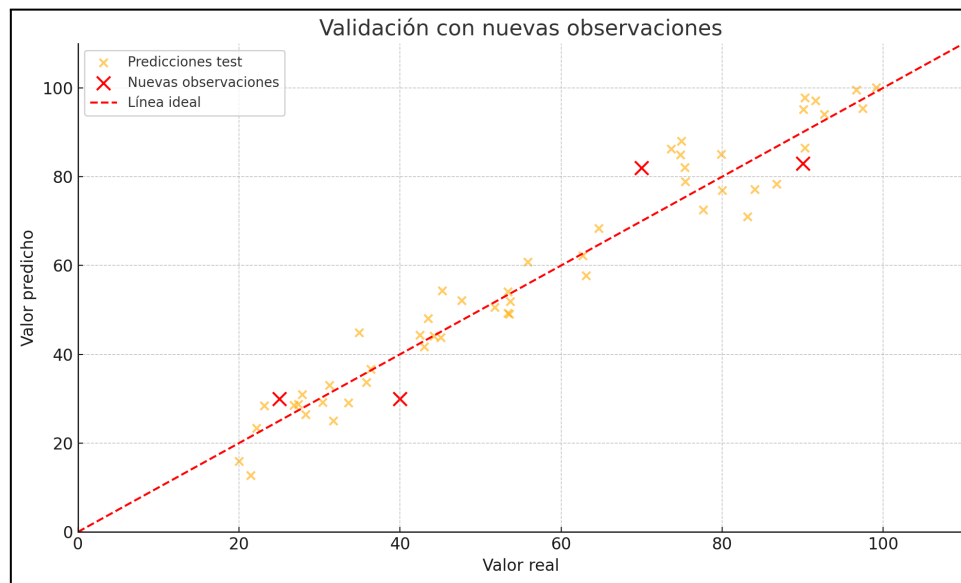
Validación con nuevas observaciones

Una prueba esencial es comprobar cómo se comporta el modelo ante nuevos datos. Esto incluye tanto observaciones normales como casos extremos o situaciones poco frecuentes. Si el modelo sigue ofreciendo predicciones razonables, es señal de que ha aprendido patrones generales. En cambio, si las predicciones se desvían mucho, especialmente en casos nuevos o inesperados, puede indicar que el modelo solo ha aprendido a funcionar bien con los datos que ya conocía. Esta

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



validación externa, aunque no siempre se incluye en los primeros ciclos de entrenamiento, es clave para asegurar que el modelo puede aplicarse en contextos reales.



Algoritmos de regresión

Introducción a los algoritmos de regresión

La regresión es una de las tareas principales dentro del aprendizaje supervisado, y su objetivo es predecir un valor numérico continuo a partir de una o más variables de entrada. A diferencia de la clasificación, que asigna etiquetas discretas, la regresión se utiliza para estimar cantidades como el precio de un producto, la duración de un trayecto o el consumo energético de una vivienda.

Aunque existe una gran variedad de algoritmos capaces de realizar tareas de regresión, en este módulo solo estudiaremos en detalle uno propio de este tipo de problemas: la regresión lineal. El resto de algoritmos que ya se han trabajado en el bloque de clasificación también pueden utilizarse para regresión, con ligeras adaptaciones. Es el caso de los árboles de decisión, los modelos de ensamble como Random Forest y Gradient Boosting, o los modelos basados en máquinas de vectores soporte (SVM), que se pueden ajustar para regresión mediante SVR (*Support Vector Regression*).

Además, existen otros algoritmos diseñados exclusivamente para regresión, como los modelos polinómicos, la regresión por mínimos absolutos o técnicas específicas para series temporales. No se abordan en este módulo, pero conocer su existencia puede ayudar a elegir mejor la estrategia en función del problema concreto.

Entender cómo funcionan estos algoritmos y cuándo utilizar cada uno es clave para construir modelos predictivos fiables. Un modelo de regresión no solo debe ofrecer buenas predicciones, sino también generalizar correctamente y ser interpretable en su contexto de uso.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Regresión lineal simple y múltiple, mínimos cuadrados y regularización

Idea principal y ejemplos de uso

La regresión lineal es un modelo supervisado que predice valores numéricos continuos a partir de una o varias variables de entrada. Asume que existe una relación lineal entre las variables independientes y la variable dependiente, y busca encontrar la mejor línea recta (o hiperplano en caso múltiple) que minimice la diferencia entre los valores predichos y los reales.

Funciona bien con datos numéricos, sin valores extremos muy desviados y donde la relación entre variables sea aproximadamente lineal. Puede aplicarse tanto a conjuntos de datos pequeños como grandes, siempre que las variables estén correctamente preparadas y escaladas si es necesario.

Algunos ejemplos habituales son la predicción de precios (como viviendas, productos o acciones), la estimación de tiempos (como el de entrega o reparación), o la proyección de cantidades continuas (como el gasto mensual de un hogar, la temperatura de un sistema o el número de ventas en función de la inversión en marketing).

Cómo funciona

La regresión lineal, tanto simple como múltiple, se basa en ajustar una línea recta (o un plano en espacios de más dimensiones) que relacione las variables independientes con la variable dependiente. El modelo aprende esa relación calculando los coeficientes que mejor explican los datos observados.

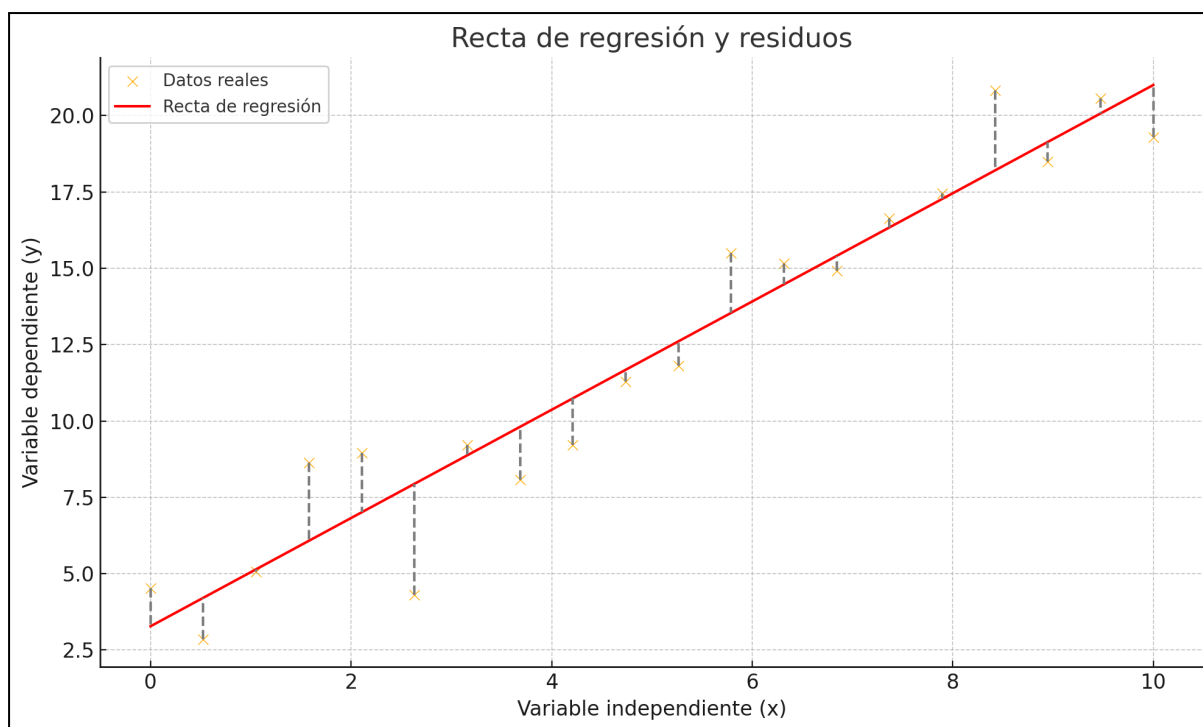
En la regresión lineal simple, se utiliza una sola variable independiente para predecir una dependiente. En la múltiple, se usan varias variables independientes. En ambos casos, el modelo estima el valor de salida combinando los valores de entrada mediante una fórmula lineal.

Durante el entrenamiento, el modelo compara sus predicciones con los valores reales y calcula la diferencia entre ellos, llamada residuo. Cuanto más pequeños y equilibrados sean esos residuos, mejor se ajusta el modelo. Por eso, el objetivo del aprendizaje es minimizar la suma de los residuos al cuadrado, es decir, que las predicciones estén lo más cerca posible de los valores reales sin errores sistemáticos.

En la siguiente imagen se puede observar una representación gráfica de una regresión lineal simple. La línea representa la predicción del modelo, mientras que los puntos muestran los datos reales. Las líneas verticales conectan cada punto con la recta y representan los residuos, es decir, el error entre el valor real y el estimado por el modelo.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)





Puntos clave del algoritmo

- La regresión lineal parte de la suposición de que existe una relación lineal entre las variables independientes y la variable dependiente
- Utiliza los residuos (diferencias entre valores reales y predichos) para ajustar el modelo, buscando minimizar su suma al cuadrado
- Es muy eficiente con datos numéricos continuos y escalados, y funciona mejor cuando las variables no están muy correlacionadas entre sí
- Permite interpretar con facilidad el impacto de cada variable independiente sobre la predicción, gracias a los coeficientes del modelo
- Es sensible a valores atípicos, ya que estos pueden desviar la recta de ajuste y empeorar el rendimiento global
- Supone ciertas condiciones como que la relación entre variables sea lineal, que los errores (residuos) sigan una distribución normal, que su variación sea constante en todos los niveles (homoscedasticidad) y que no haya patrones entre los errores (ausencia de autocorrelación).

Cuándo funciona bien y cuándo no

Funciona bien cuando la relación entre las variables es realmente lineal, no hay valores atípicos extremos y se cumplen las condiciones estadísticas básicas como normalidad de los residuos, homoscedasticidad y ausencia de autocorrelación. La linealidad se tiene que dar entre cada variable independiente y la variable dependiente, no entre las variables independientes entre sí. Es especialmente útil cuando se busca una interpretación clara del impacto de cada variable sobre el resultado.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



No funciona bien cuando hay relaciones no lineales, muchas variables irrelevantes o correlacionadas entre sí (multicolinealidad), o cuando hay ruido excesivo en los datos. También puede fallar si los residuos muestran patrones o si su variación cambia a lo largo de los valores predichos.

Particularidades del algoritmo

Una característica propia de la regresión lineal es el uso del método de los mínimos cuadrados. Esta técnica ajusta la recta o el hiperplano buscando los coeficientes que minimizan la suma de las diferencias al cuadrado entre los valores reales y los predichos. Gracias a este criterio, se obtiene la mejor aproximación global a los datos de entrenamiento. Por ejemplo, en un modelo que predice la temperatura en función del número de horas de sol, la recta se ajusta intentando minimizar los errores entre las temperaturas reales y las estimadas por el modelo.

Además, la regresión lineal parte de una serie de condiciones estadísticas que deben cumplirse para que el modelo sea válido. Una de ellas es la linealidad. Debe existir una relación lineal entre cada variable independiente y la variable dependiente. Esta relación debe mantenerse entre todas las variables independientes respecto a la variable a predecir, no solo entre algunas. En un caso como predecir el precio de una vivienda a partir de su tamaño, antigüedad y número de habitaciones, cada una de estas variables debería tener un comportamiento lineal con respecto al precio para que el modelo funcione correctamente.

Otra suposición clave es la normalidad de los residuos. Esto significa que las diferencias entre los valores reales y los predichos deben seguir una distribución normal. Cumplir este requisito permite interpretar mejor los resultados del modelo y aplicar ciertas técnicas estadísticas con más garantías.

También es importante que exista homoscedasticidad. Esto quiere decir que la varianza de los residuos se mantenga constante para todas las predicciones. Si la dispersión de los errores cambia según el valor predicho, el modelo puede estar cometiendo errores más grandes en ciertos rangos, lo que se conoce como heteroscedasticidad. Por ejemplo, si al predecir ingresos según nivel educativo, los errores se disparan en los niveles más altos, esto sería un caso de heteroscedasticidad.

La ausencia de autocorrelación en los residuos es otra condición necesaria. Los errores cometidos por el modelo no deberían seguir ningún patrón ni estar relacionados entre sí, especialmente si los datos están ordenados temporalmente. Si los residuos presentan autocorrelación, como podría ocurrir al predecir el número de visitas diarias a una web, puede significar que hay comportamientos repetitivos no captados por el modelo.

En la regresión múltiple hay una particularidad adicional. Es fundamental evitar la multicolinealidad, que ocurre cuando dos o más variables independientes están muy correlacionadas entre ellas. Esto dificulta saber qué variable está influyendo realmente en la predicción y puede hacer que los coeficientes del modelo cambien bruscamente con pequeñas variaciones en los datos. Por ejemplo, en un modelo que predice el consumo energético a partir del número de electrodomésticos y el tamaño del hogar, estas dos variables podrían estar tan relacionadas que una de ellas no aporte información nueva.

Por último, en los modelos con muchas variables independientes, es clave seleccionar bien cuáles se van a incluir. Deben escogerse las más relacionadas con la variable dependiente y, al mismo tiempo, que no estén demasiado correlacionadas entre sí. Para ello pueden usarse técnicas como la matriz

de correlación, métodos de selección automática o reducción de dimensionalidad como el análisis de componentes principales (PCA).

Support Vector Regression, margen ϵ y kernels

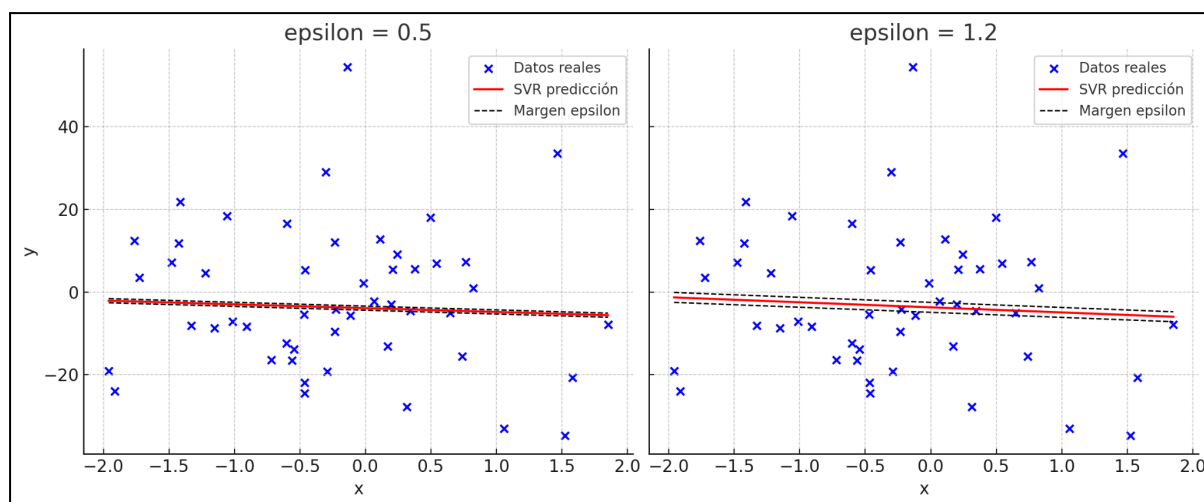
VR es una técnica para hacer predicciones numéricas basada en la misma idea que las máquinas de soporte vectorial usadas en clasificación. Pero en lugar de separar clases, SVR busca ajustar una línea o curva que pase lo más cerca posible de los datos sin preocuparse por errores pequeños.

Para eso se define un margen de tolerancia llamado **epsilon (ϵ)**. Si la predicción cae dentro de ese margen con respecto al valor real, no se considera un error importante. Solo se tienen en cuenta los casos que se desvían mucho, y son esos los que influyen en la construcción del modelo. Esto hace que el modelo sea menos sensible a pequeñas fluctuaciones y más robusto frente al ruido.

En la siguiente imagen, a la izquierda, con un epsilon de 0.5, el modelo ajusta una línea más precisa. El margen es estrecho, por lo que muchos puntos quedan fuera y se consideran errores importantes que el modelo intenta corregir. Esto puede hacer que se adapte más a los datos, pero también corre más riesgo de sobreajuste.

A la derecha, con un epsilon más amplio (1.2), el modelo permite mayor desviación sin penalizarla. El margen abarca más puntos, que se consideran aceptables aunque no estén exactamente en la línea. Esto da lugar a una predicción más simple y robusta frente al ruido.

Esta visualización ayuda a entender cómo epsilon actúa como un "colchón" de tolerancia en SVR, decidiendo qué errores se ignoran y cuáles afectan directamente al modelo.



El comportamiento del modelo depende de varios hiperparámetros. Epsilon marca cuánta desviación se tolera sin penalizar. C indica cuánto se penalizan los errores grandes: valores bajos dan lugar a modelos más simples y valores altos fuerzan un mayor ajuste. Kernel transforma los datos si la relación no es lineal. El kernel más usado en regresión es el radial o RBF, pero también se puede usar el lineal o el polinómico. Si el kernel es no lineal, se añade el parámetro gamma, que controla la influencia de cada punto sobre la predicción.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



SVR tiene otras particularidades importantes. Es sensible a la escala de los datos, por lo que es imprescindible aplicar un escalado previo a las variables antes de entrenar. También hay que tener en cuenta que, aunque puede manejar relaciones complejas, no escala bien con conjuntos de datos muy grandes. Cuantos más vectores soporte necesite, más costoso será el entrenamiento.

Por último, hay que destacar que no es un modelo interpretable. Si se usa un kernel no lineal, es difícil entender cómo influyen las variables en la predicción final. Por eso, SVR es recomendable cuando lo que se busca es precisión más que explicación.

Árboles de decisión para regresión, partición de varianza

Un árbol de decisión para regresión divide el espacio de datos en regiones más pequeñas aplicando reglas sucesivas basadas en los valores de las variables independientes. A diferencia de la clasificación, donde el objetivo es asignar clases, en regresión cada hoja del árbol devuelve un valor numérico, normalmente la media de los valores reales que han llegado hasta ella.

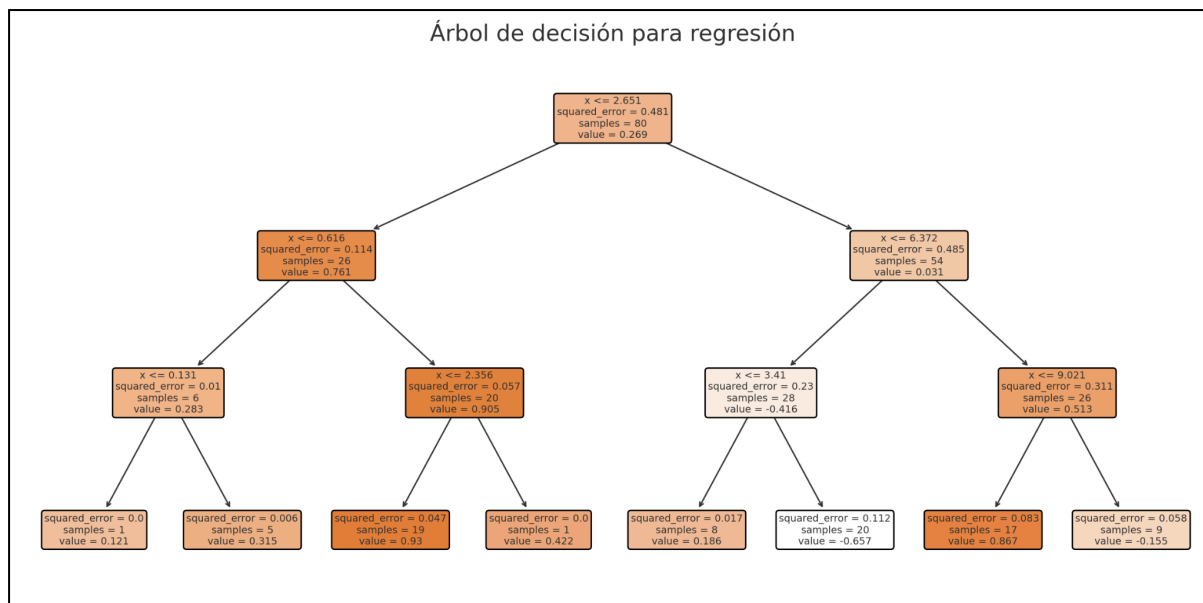
El criterio de división más habitual es la reducción de varianza. En cada nodo, el algoritmo elige la variable y el punto de corte que generan dos subconjuntos con menor varianza interna, es decir, grupos donde los valores de salida son más parecidos entre sí. Esto indica que la división ha separado correctamente los datos según el valor a predecir, y mejora la precisión del modelo en cada hoja.

Este tipo de modelo no requiere que exista una relación lineal entre las variables ni que los datos sigan una distribución concreta. Puede capturar relaciones no lineales, efectos de umbral y comportamientos abruptos que otros modelos, como la regresión lineal o una SVR con kernel lineal, no detectan fácilmente.

Sin embargo, los árboles de regresión tienden a sobreajustar si no se aplican restricciones. Para evitarlo, se recomienda limitar la profundidad máxima del árbol (`max_depth`) o fijar un número mínimo de muestras por hoja (`min_samples_leaf`). Por ejemplo, valores entre 3 y 7 para la profundidad y entre 5 y 20 muestras por hoja suelen ser eficaces en conjuntos pequeños o medianos.

Una diferencia importante respecto a clasificación es que aquí se utilizan métricas continuas como el error cuadrático medio (MSE) para evaluar el rendimiento, y no precisión o matrices de confusión. Si el modelo está bien ajustado, las predicciones siguen el patrón real de forma escalonada, según las regiones que ha generado.

En la siguiente imagen se muestra cómo el modelo construye divisiones sobre la variable independiente (eje horizontal) y asigna una predicción constante a cada segmento. Cada valor predicho corresponde a la media de los datos reales en esa región. Esta representación visual permite entender cómo un árbol de regresión aproxima funciones no lineales mediante bloques con valores constantes.



Random Forest para regresión, ensamble y reducción de varianza

Random Forest es un modelo de ensamble que combina múltiples árboles de decisión para obtener predicciones más robustas y precisas. Cada árbol se entrena con una muestra aleatoria del conjunto de entrenamiento (con reemplazo) y, en el caso de regresión, la predicción final se calcula haciendo la media de las predicciones individuales de todos los árboles.

Este enfoque reduce la varianza del modelo sin aumentar demasiado el sesgo. Aunque un solo árbol puede sobreajustarse fácilmente a los datos, la media de muchos árboles independientes compensa estas desviaciones y mejora la capacidad de generalización. Esta estrategia hace que Random Forest funcione bien incluso en presencia de ruido o datos no lineales.

En regresión, Random Forest no necesita que las variables independientes tengan una relación lineal con la variable objetivo ni que los datos sigan una distribución concreta. Funciona bien con conjuntos de datos complejos, con muchas variables o con interacciones entre ellas, y no requiere escalar las características.

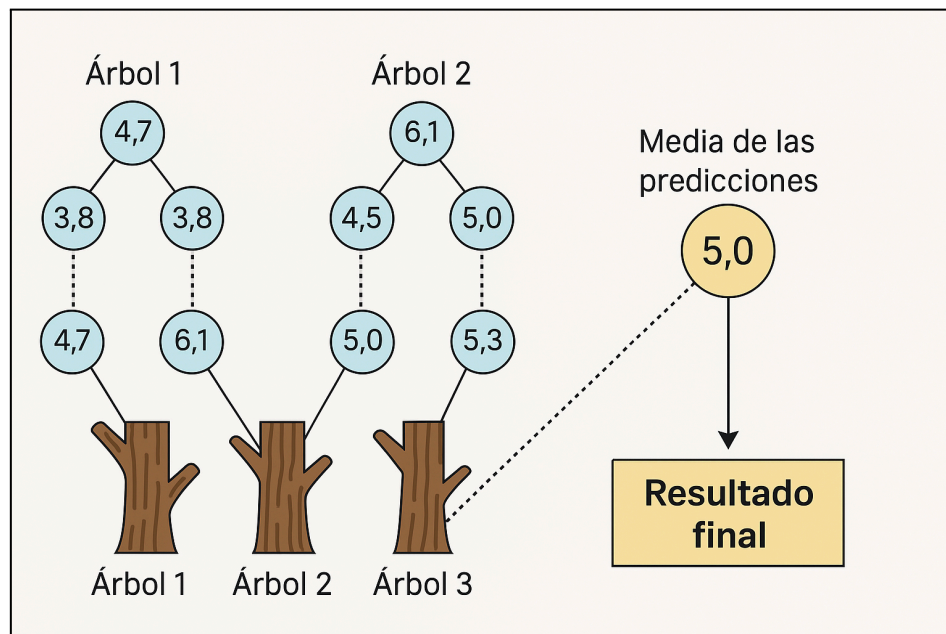
A diferencia de un árbol individual, donde se puede seguir una única ruta de decisión, aquí no hay una interpretación directa del modelo completo. Aun así, se pueden calcular importancias de las variables para saber cuáles influyen más en la predicción. También permite ajustar hiperparámetros específicos como el número de árboles (`n_estimators`), la profundidad máxima de cada árbol (`max_depth`) o el número de variables consideradas en cada división (`max_features`). Por ejemplo, `max_features='sqrt'` suele ser una opción adecuada.

En la siguiente imagen se representa el funcionamiento interno de Random Forest en un problema de regresión. Cada árbol genera una predicción distinta a partir de sus propias reglas y subconjuntos de datos. La predicción final del modelo se obtiene calculando la media de todas esas salidas

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



individuales. Este proceso reduce el sobreajuste de los árboles por separado y proporciona una estimación más estable y precisa.



Adaboost para regresión, errores ponderados continuos

AdaBoost también puede aplicarse a problemas de regresión, donde el objetivo no es clasificar, sino predecir valores numéricos continuos. En este caso, el algoritmo entrena una serie de modelos simples, normalmente árboles de regresión poco profundos, de forma secuencial. Cada modelo intenta corregir los errores cometidos por el anterior, dando más importancia a los puntos donde las predicciones han fallado más.

El proceso comienza asignando el mismo peso a todas las observaciones. Después de entrenar el primer modelo, se calculan los errores y se ajustan los pesos: los puntos con errores más grandes reciben mayor peso en el siguiente modelo. Así, cada nuevo estimador se centra más en las observaciones difíciles de predecir.

Este ajuste progresivo permite que el conjunto de modelos se adapte a zonas complejas del dataset sin necesidad de recurrir a modelos muy complejos desde el inicio. Es una estrategia eficaz para mejorar la precisión en regiones con ruido, valores atípicos o patrones irregulares, siempre que se controle adecuadamente el número de iteraciones y la complejidad del modelo base.

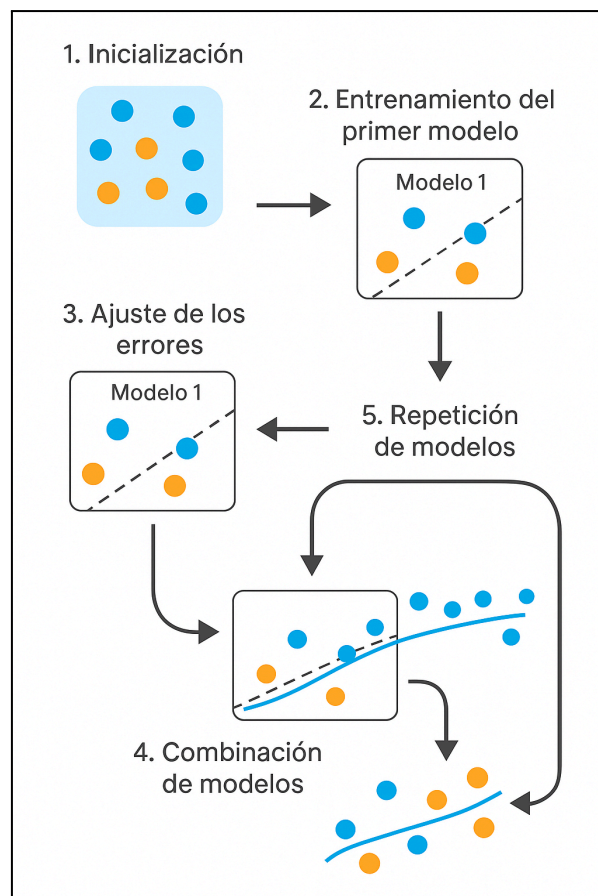
La predicción final se obtiene combinando los resultados de todos los modelos, ponderando su contribución según su rendimiento. En lugar de una votación como en clasificación, se calcula una media ponderada de los valores estimados.

Entre los hiperparámetros principales destacan el número de estimadores (`n_estimators`, por ejemplo entre 50 y 200), la tasa de aprendizaje (`learning_rate`, normalmente entre 0.01 y 1) y la función de pérdida (`loss`), que puede ser lineal, cuadrática o exponencial, en función de cómo se

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



quieran penalizar los errores. El modelo base (`base_estimator`) suele tener profundidad limitada para evitar que cada árbol sobreajuste por separado.



Comparación general de algoritmos

Algoritmo	Objetivo general	Ventajas	Limitaciones	Hiperparámetros más importantes	Puntos clave a tener en cuenta
Regresión lineal (simple y múltiple)	Ajustar una recta o hiperplano que minimice la diferencia entre los valores	Interpretación sencilla, rápido de entrenar, útil como punto de partida. Eficaz con relaciones lineales	No capta relaciones no lineales, afectada por multicolinealidad, sensible a outliers	<code>fit_intercept</code> , <code>normalize</code> (según la implementación)	Requiere relación lineal entre todas las variables independientes y la dependiente. Requiere

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



	reales y los predichos				residuos normales, homocedasticidad y sin autocorrelación
SVR (Support Vector Regression)	Ajustar una función que minimiza errores dentro de un margen ϵ , maximizando el margen y usando kernels si es necesario	Flexible con kernels, tolera relaciones no lineales. Control sobre errores mediante ϵ	Costosa computacionalmente, difícil de ajustar. Sensible a selección de hiperparámetros	C (penalización), epsilon (margen), kernel, gamma	El valor de ϵ define el margen sin penalización. Útil en datos ruidosos si se ajusta bien. Requiere escalado
Árboles de decisión	Dividir los datos en regiones homogéneas mediante reglas basadas en la variable dependiente	Captura relaciones no lineales, fácil de visualizar, no necesita normalización	Tendencia a sobreajuste sin regularización, predicción por tramos constantes	max_depth, min_samples_split, min_samples_leaf, criterion	No necesita normalización. Acepta variables mixtas. Ideal si hay umbrales o patrones no lineales evidentes
Random Forest	Promediar múltiples árboles de decisión entrenados sobre subconjuntos aleatorios para mejorar estabilidad y precisión	Reduce el sobreajuste, robusto a outliers, buen rendimiento sin mucho ajuste	Menos interpretable, más costoso. No siempre mejora tras cierto número de árboles	n_estimators, max_features, max_depth, n_jobs	Evalúa la importancia de variables. No requiere normalización. Entrenamiento más largo, pero paralelo

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



AdaBoost	Combinar modelos simples corrigiendo iterativamente los errores de predicción con más peso en los más grandes	Mejora modelos simples, adapta bien en zonas difíciles, precisa con pocos hiperparámetros	Muy sensible a outliers, entrenamiento secuencial, no paralelizable	n_estimators, learning_rate, loss, base_estimator	Funciona bien si se ajustan iteraciones y modelo base. Penaliza más los errores altos si se ajusta la función de pérdida
----------	---	---	---	---	--